APPLICATION STUDY: DNA SEQUENCE ALIGNMENT

2026 SPRING ESD

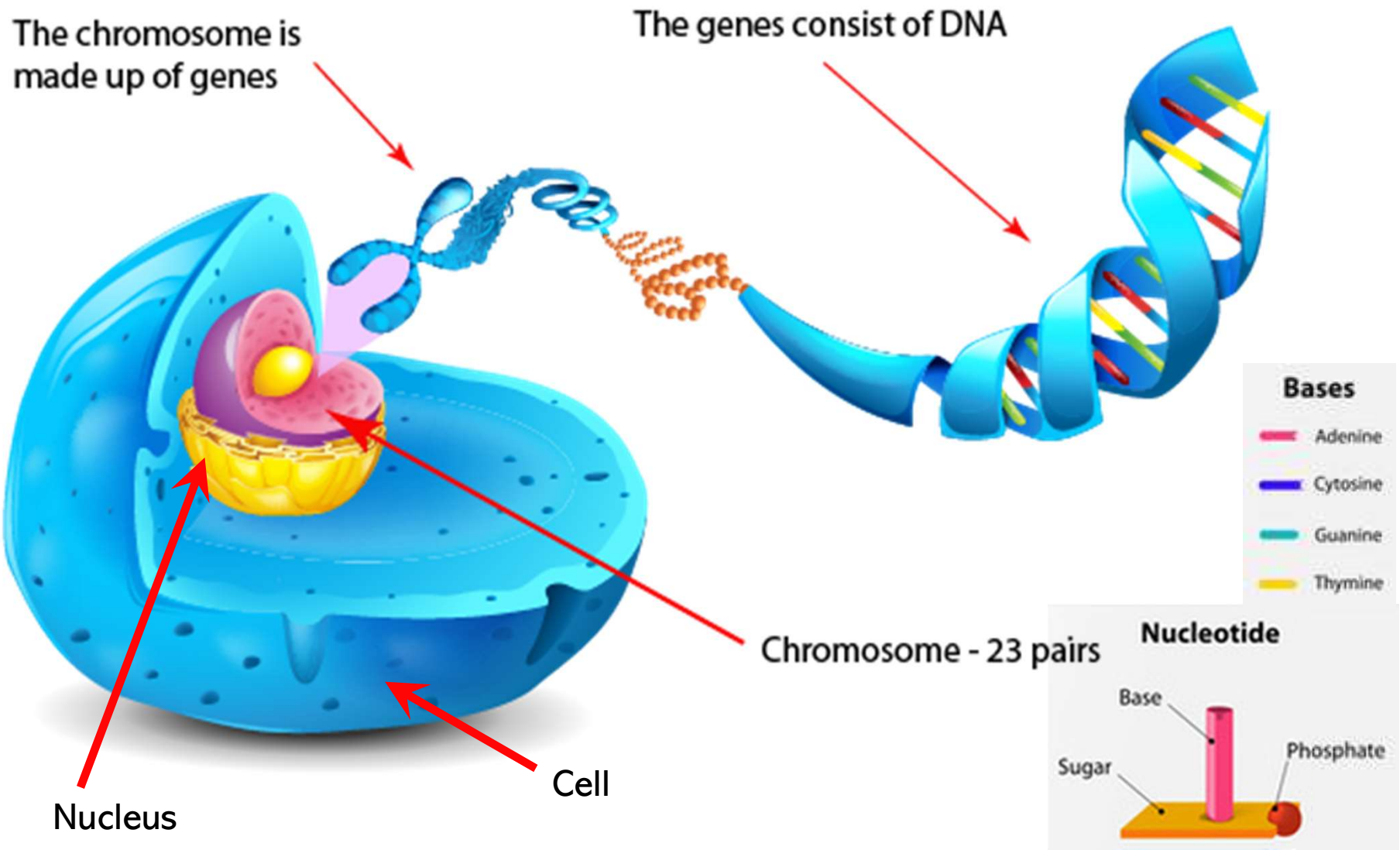
JINHO LEE

ECE, SNU

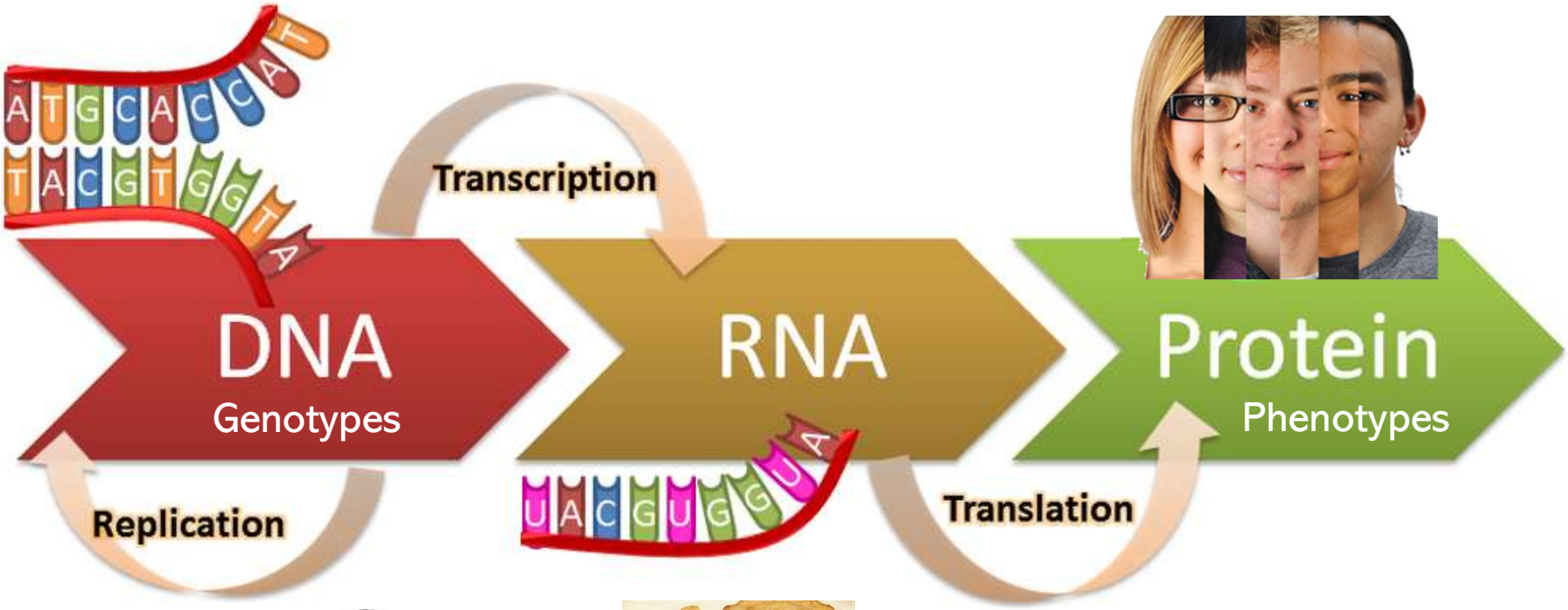
CONTENT

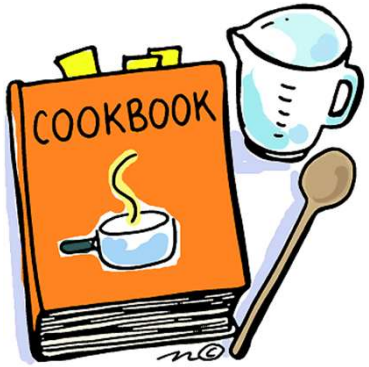
- DNA
- Sequence (Text) alignment
- Smith-Waterman algorithm
  - Recursive
  - Dynamic programming
  - Affine gap model
- Parallelization strategies

HAVE YOU HEARD OF THE HUMAN GENOME PROJECT?

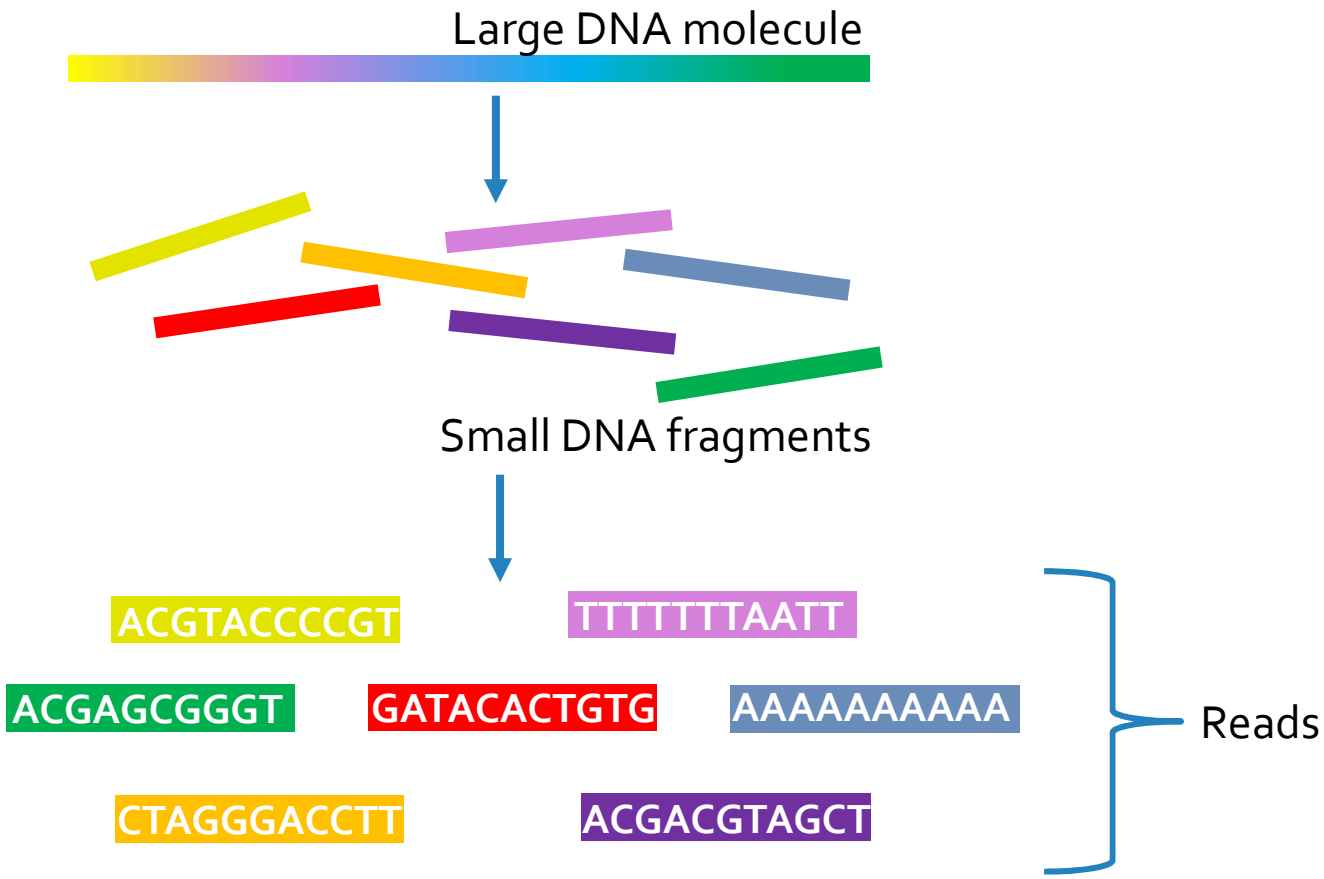


DNA IS THE SOURCE CODE FOR THE LIVING



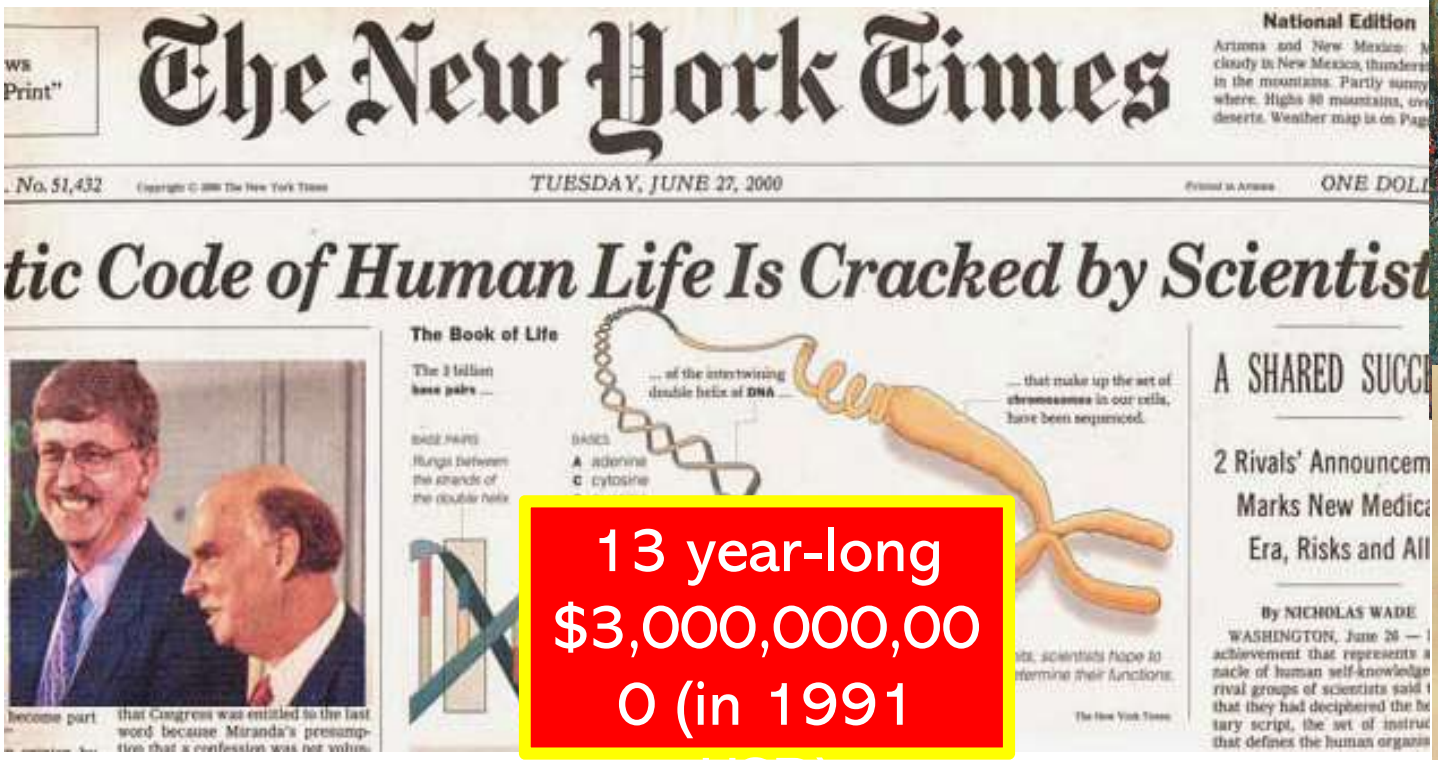


GENOME SEQUENCING



HUMAN GENOME PROJECT

- A project that tries to find the entire DNA base pairs of human being
- 1990-2003


The New York Times

National Edition

No. 51,432 TUESDAY, JUNE 27, 2000 ONE DOL

tic Code of Human Life Is Cracked by Scientist

The Book of Life

... of the intertwining double helix of DNA ...

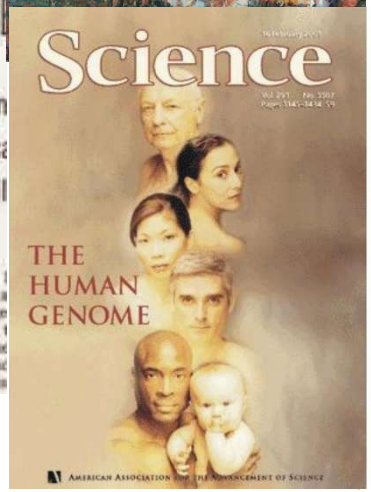
... that make up the set of chromosomes in our cells, have been sequenced.

A SHARED SUCC

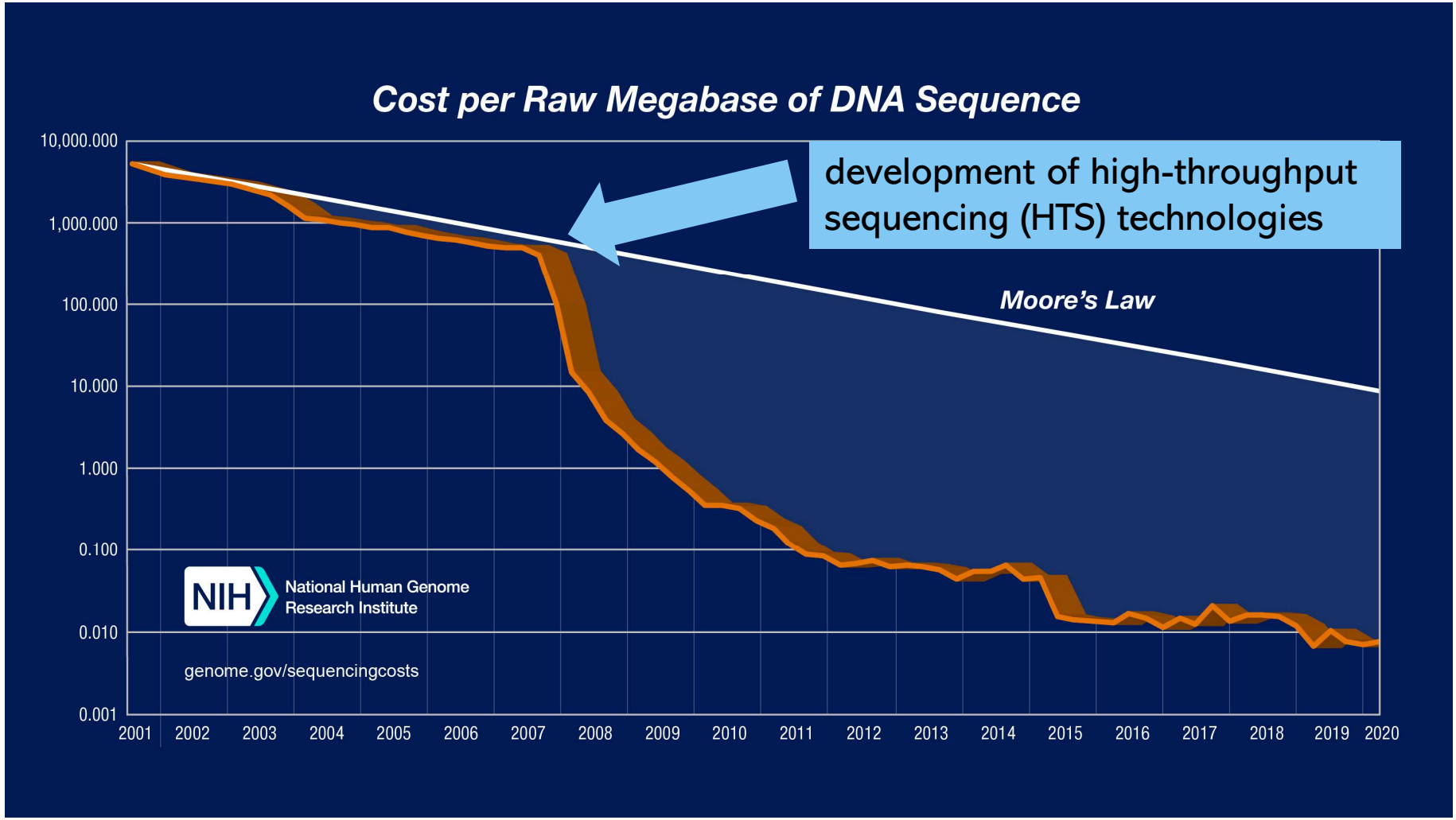
2 Rivals' Announcem Marks New Medica Era, Risks and All

By NICHOLAS WADE

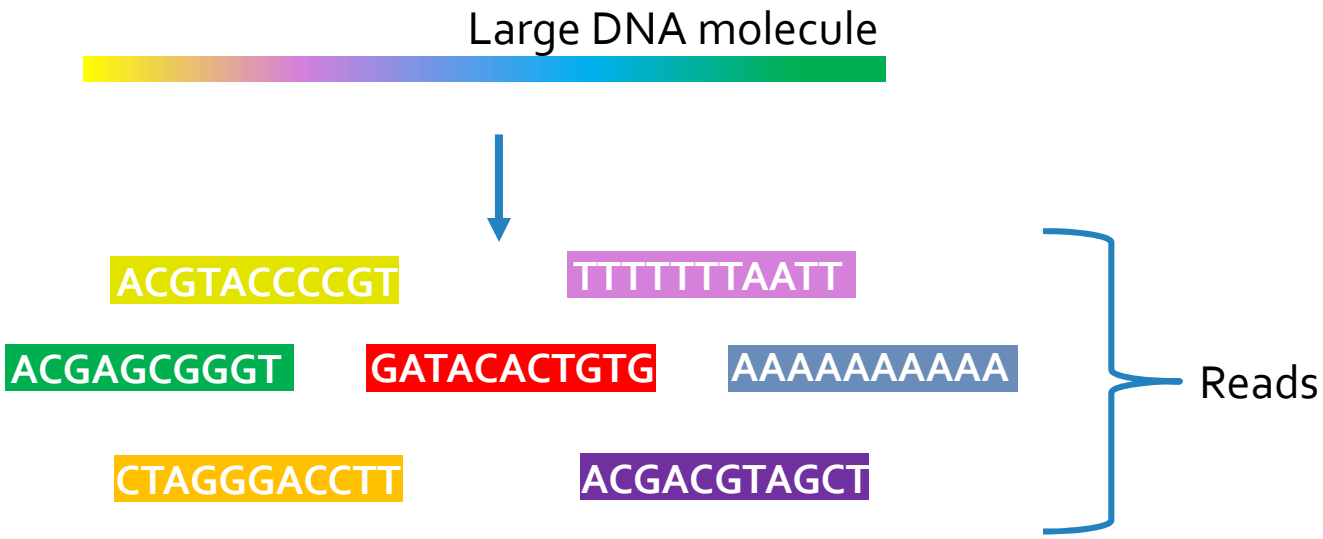
13 year-long
$3,000,000,00
0 (in 1991



COST OF SEQUENCING DROPS RAPIDLY



GENOME SEQUENCING



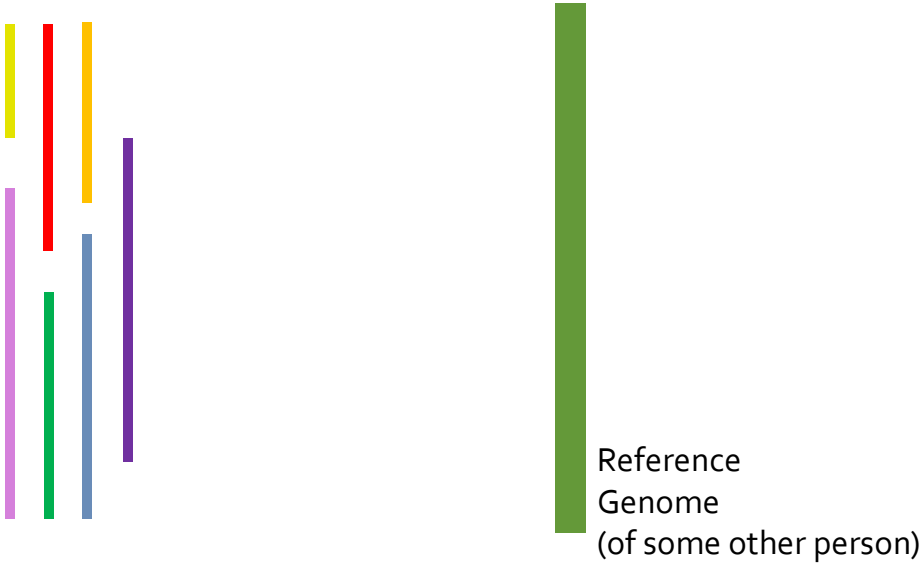
- Can you tell the original sequence from these short pieces??

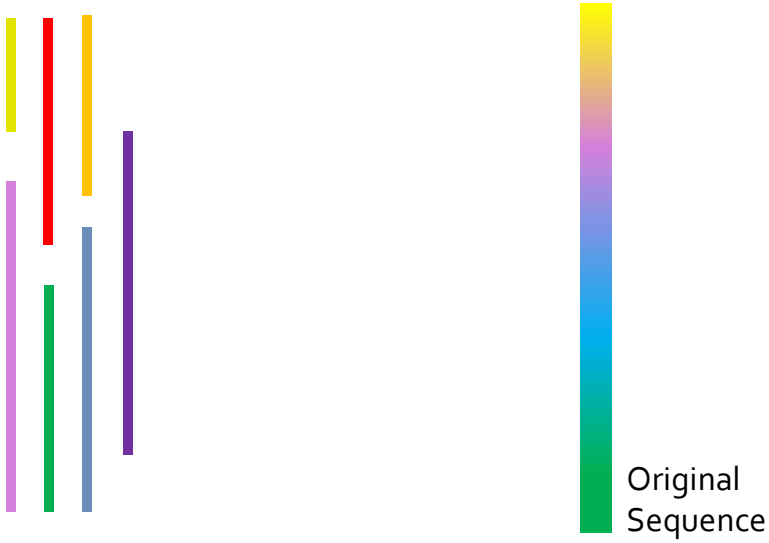
1. Have a 1k-page book
2. Let 1m friends transcribe them individually
3. Cut them all into random ~10 page pieces
4. Give them all to another friend "Hey, restore the original book"

GENOME SEQUENCE ANALYSIS

Read Mapping, method of aligning the reads against a known reference genome to **detect matches and variations.**

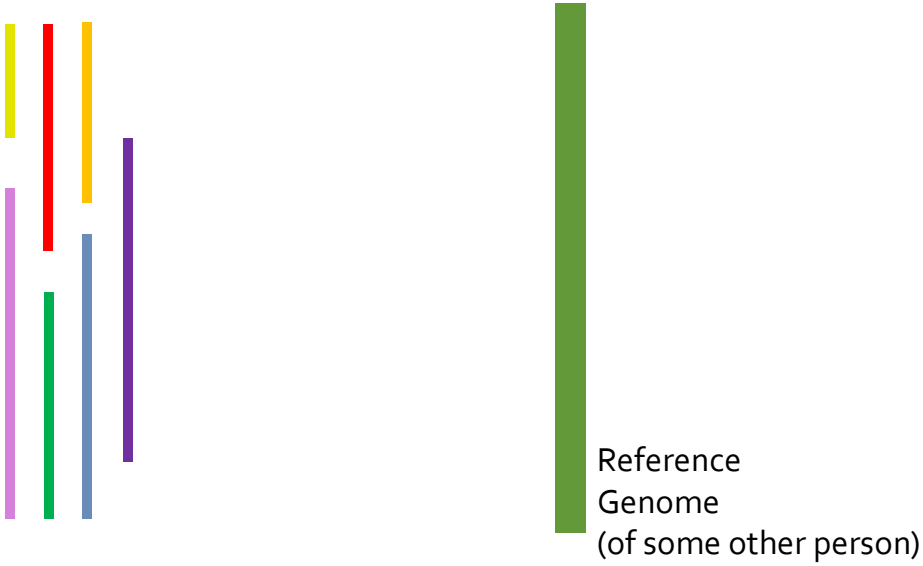
***De novo* Assembly,** method of merging the reads in order to **construct** the original sequence.





GENOME SEQUENCE ANALYSIS

Read Mapping, method of aligning the reads against a known reference genome to **detect matches and variations.**



- A text matching problem
- Only four characters: A, G, C, T

PROBLEM

- DNAs are mostly similar, but different everywhere

...ACATGCCGACATTTCATAGGCC...

...ACATGTCGACATTTCATAAGCC...

- We need approximate matching

DIFFICULTY OF APPROXIMATE MATCHING

- Not only the characters can be different, but there can be gaps

```
CGACATT
CGACATT
```

matches

```
CGACATT
CGAGATT
```

mismatches

```
CGACATT
CGA-ATT
```

deletions

```
CGAC-ATT
CGACGATT
```

insertions

WHICH IS THE BEST MAPPING?

- Try to map this

```
Ref  : c g g g t a t c c a a
Read : c c c t a g g t c c c a

Ref  : c g g g t a - - t - c c a a
Read : c c c - t a g g t c c c - a

Ref  : c g g g t a - - - t c c a a
Read : c c - - c t a g g t c c c a

Ref  : c - g g g t a - - t c c a a
Read : c c - - c t a g g t c c c a
```

SMITH-WATERMAN ALGORITHM

- There is a score matrix (example)

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

- Score does not go down below zero
  - Meaning: you can start from anywhere

SMITH-WATERMAN ALGORITHM

- There is a score matrix (example)

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

- ‘gap’ penalty for Insertions and deletions:
  - Example: -5
- Start counting from the beginning
- Score does not go down below zero
  - Meaning: you can start from anywhere

SMITH-WATERMAN ALGORITHM

- There is a score matrix (example)

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2

- 'gap' penalty for Insertions and deletions:
  - Example: -5
- Start counting from the beginning
- Score does not go down below zero
  - Meaning: you can start from anywhere
- Find the maximum
  - Meaning: you can end anywhere

```
Ref  : AAG
Read : AGC
       200

Ref  : AAG-
Read : -AGC
       0240

Ref  : AAG-
Read : A-GC
       2020

Ref  : AAG-
Read : -AGC
       0240
```

HOW DO YOU CONSIDER ALL POSSIBILITIES?

- Recursive structure

```
Ref  : AAG
Read : AGC
```

1. Consult the score table

```
Ref  : A  AG
       +
Read : A  GC
       2
```

2. Insertion

```
Ref  : -  AAG
       +
Read : A  GC
       0
```

3. deletion

```
Ref  : A  AG
       +
Read : -  AGC
       0
```

$$F(0,0) = 0$$

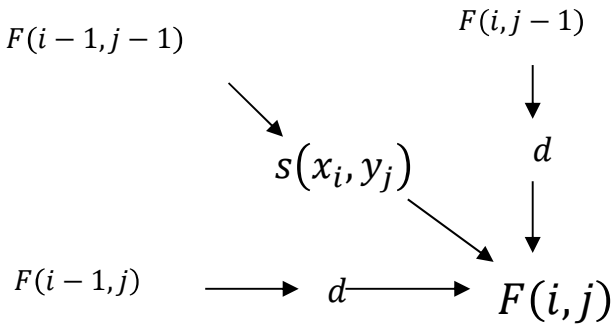
$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!

The length is around 100~10000 according to the sequencing device

		A	A	G
A				
G				
C				



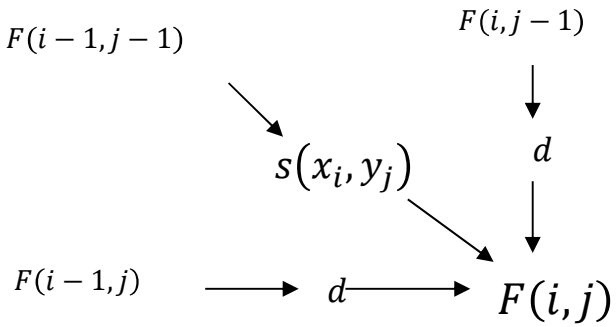
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0			
G	0			
C	0			

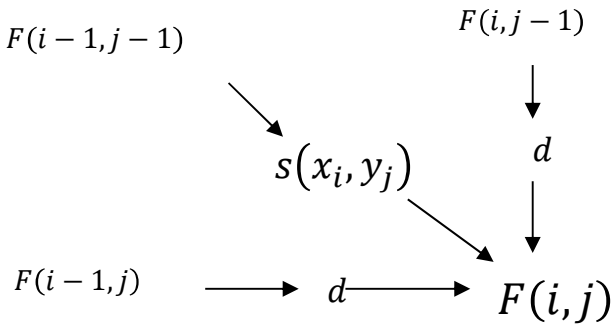
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0
  - Start calculating

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0	2 -5 -5 0		
G	0			
C	0			

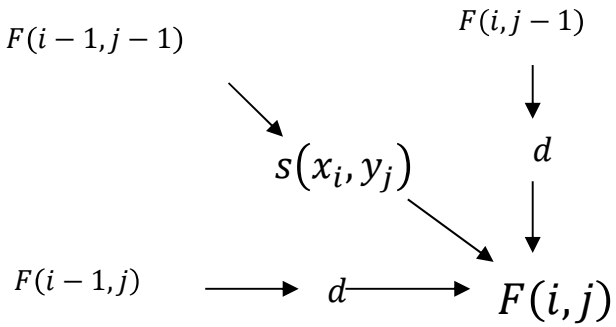
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0
  - Start calculating

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0	2		
G	0			
C	0			

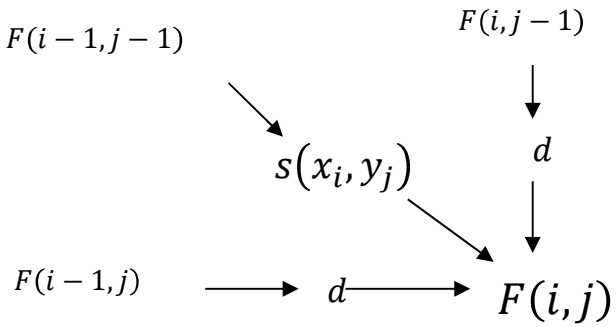
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0
  - Start calculating

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0	2		
G	0	0		
C	0	0		

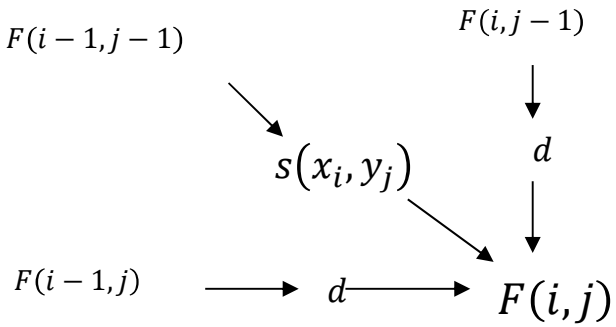
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0
  - Start calculating

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0		
C	0	0		

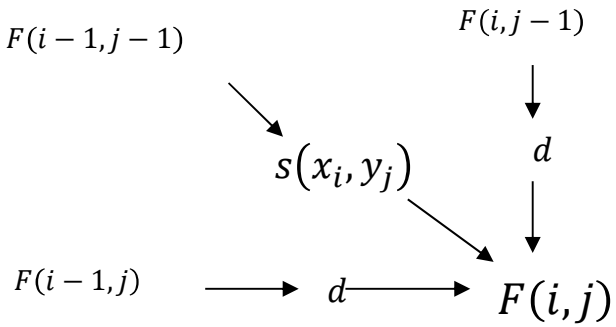
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0
  - Start calculating

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0	0	4
C	0	0		

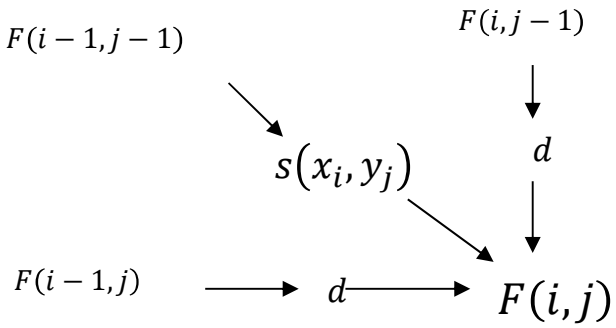
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Dynamic programming!
  - Set the boundaries to be 0
  - Start calculating

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0	0	4
C	0	0	0	0

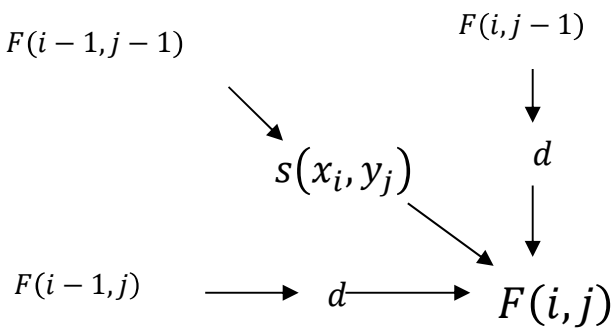
$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Traceback
  - Store the best position
  1. **Follow F[] to find parent**
     - Less memory, more computation
  2. Keep the direction matrix of each cell during filling
     - More memory, less computation

		A	A	G
	0	0	0	0
A	0	2	2	0
G	0	0	0	4
C	0	0	0	0

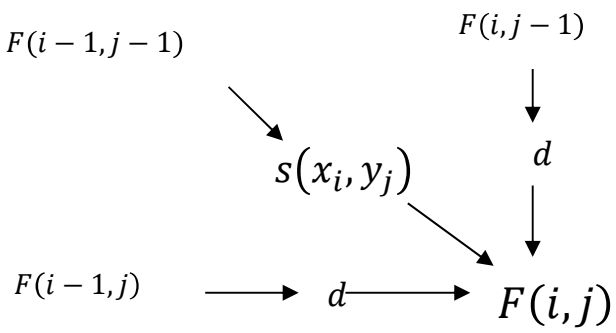


$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

WE KNOW WHAT TO DO WITH THIS (PROBABLY?)

- Traceback
  - Store the best position
  1. Follow F[] to find parent
     - Less memory, more computation
  2. **Keep the direction matrix of each cell during filling**
     - More memory, less computation



		A	A	G
	0	0	0	0
A	0			
G	0			
C	0			

$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

SERIAL VERSION

- After initialization, calculate row by row

```
For r in rows:
    For c in cols:
        F[r][c] = max(…)
```

		A	A	G
	0	0	0	0
A	0	①		
G	0	②		
C	0	③		

$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

PARALLEL VERSION

- Cool!

```
#pragma omp parallel for
For r in rows:
    For c in cols:
        F[r][c] = max(…)
```

		A	A	G
	0	0	0	0
A	0	①		
G	0	②		
C	0	③		

$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

NOT SO FAST..

- Strict dependency between cells..

```
#pragma omp parallel for
For r in rows:
    For c in cols:
        F[r][c] = max(…)
```

		A	A	G
	0	0	0	0
A	0			
G	0			
C	0			

$$F(0,0) = 0$$

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) \\ F(i-1,j) + d \\ F(i,j-1) + d \\ 0 \end{cases}$$

PARALLEL VERSION TAKE 2

- Let lower rows follow the upper rows

- How do we know if some cell is finished?

		A	A	G
	0	0	0	0
A	0			①
G	0		②	
C	0	③		

PARALLEL VERSION TAKE 2

- Let lower rows follow the upper rows

- How do we know if some cell is finished?

		A	A	G
	0	0	0	0
A	0	Done	Done	Not
G	0	Done	Not	Not
C	0	Not	Not	Not

Done flag matrix

		A	A	G
	0	0	0	0
A	0			①
G	0		②	
C	0	③		

	Col
R1	7
R2	4
R3	3

Progress counter

PARALLEL VERSION TAKE 2

- Let lower rows follow the upper rows
- How do we know if some cell is finished?

		A	A	G
	0	0	0	0
A	0			①
G	0		②	

Our lesson so far:
Synchronization = Bad
Bad = Synchronization

	0	0	0	0
A	0	Done	Done	Not
G	0	Done	Not	Not
C	0	Not	Not	Not

Done flag matrix

	Col
R1	7
R2	4
R3	3

Progress counter

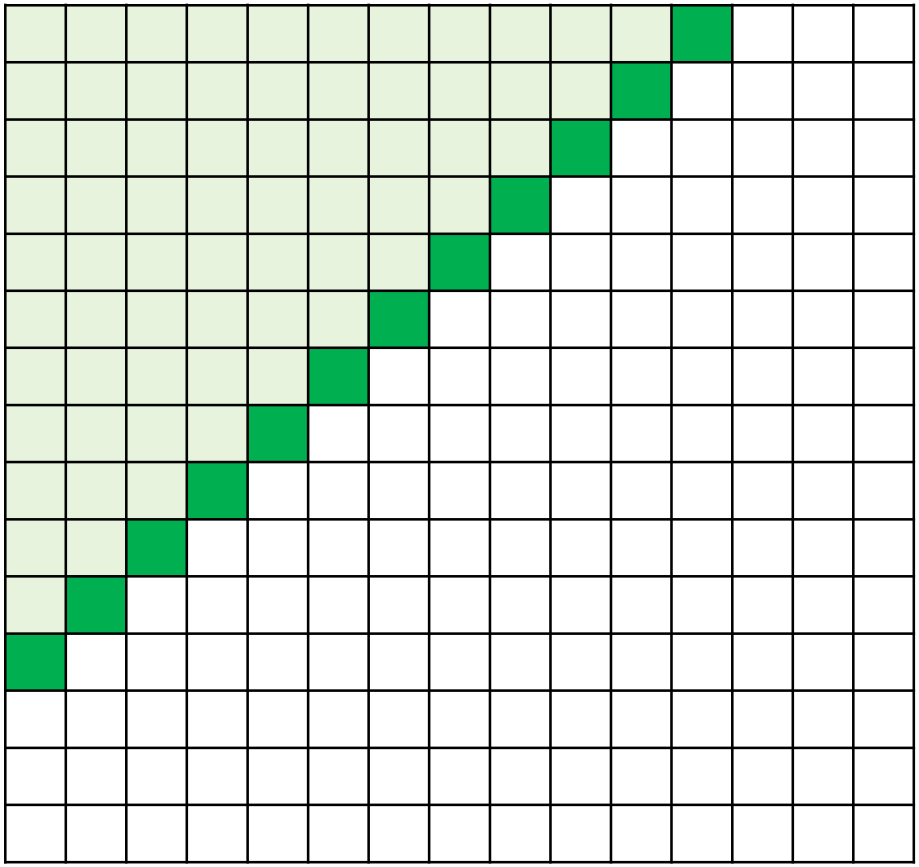
PARALLEL VERSION TAKE 3

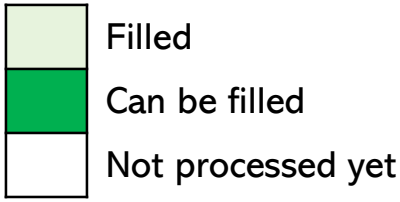
- Anti-diagonal traversal!

```
For d in anti-diagonals:
    For e in d.elements:
        (r,c) = convert_de_to_rc(d,e)
        F[r][c] = max(…)
```

		A	A	G
	0	0	0	0
A	0	①		
G	0	②		
C	0	③		

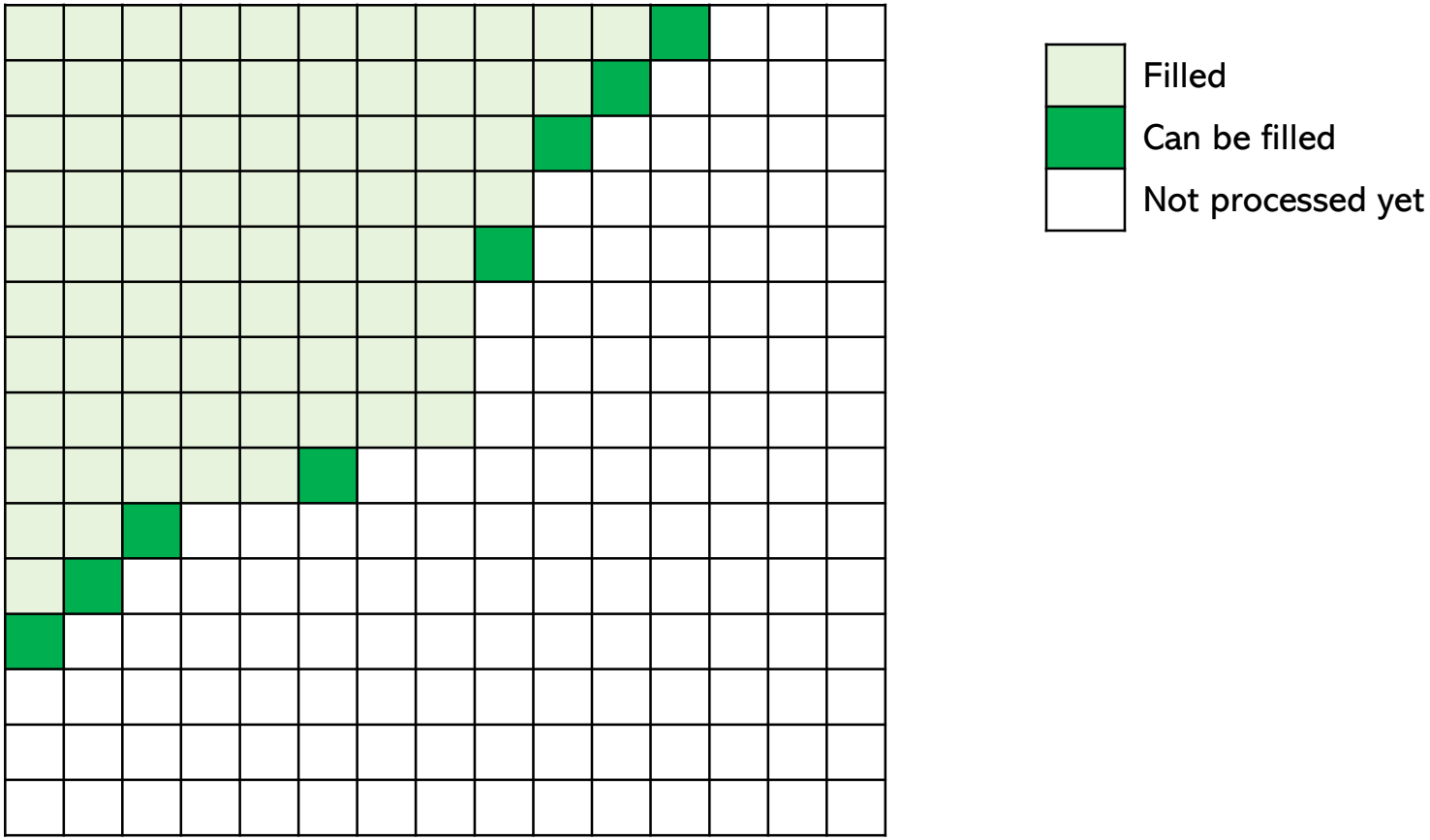
ANTI-DIAGONAL PARALLELISM





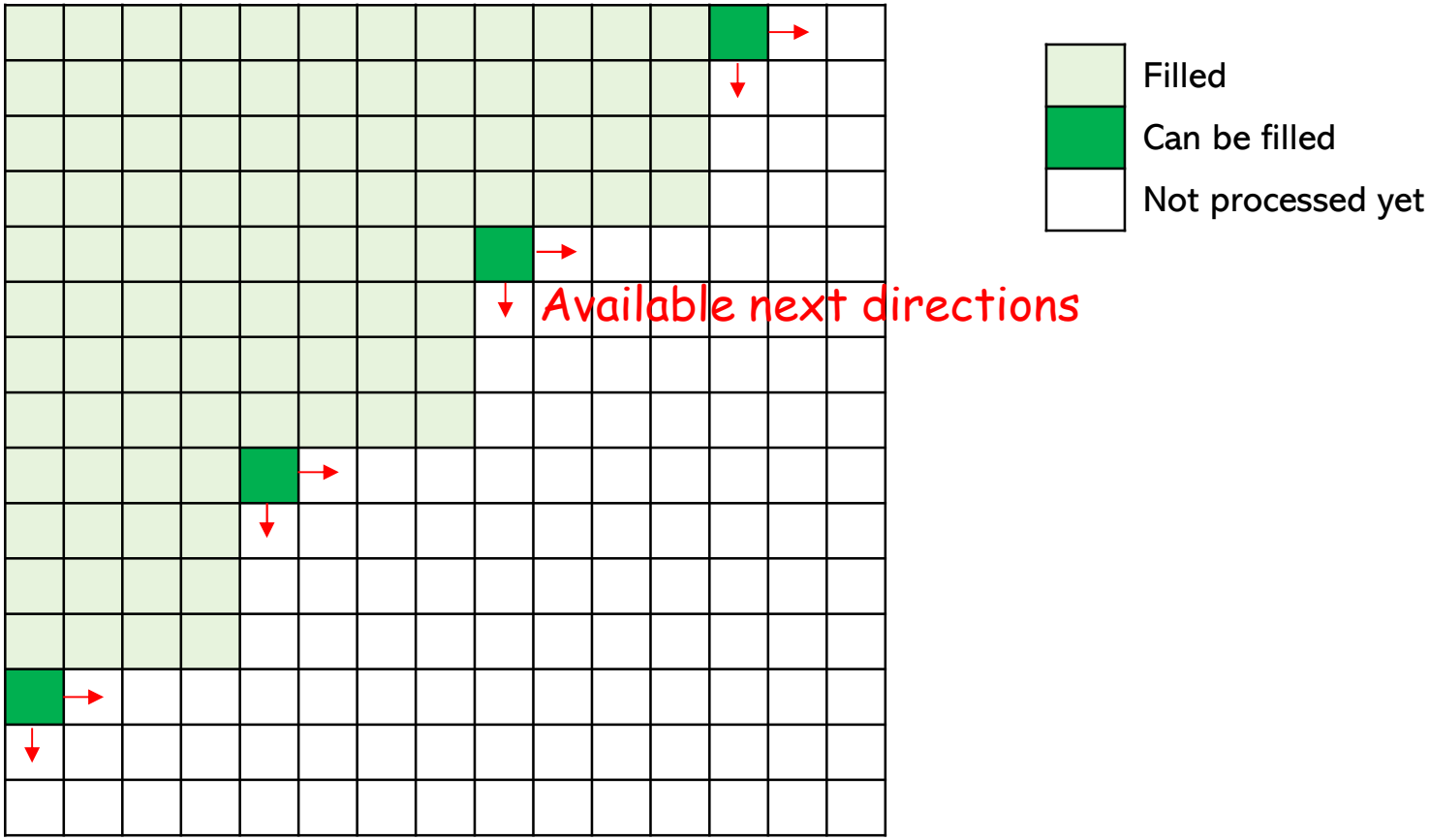
- Parallelism exists along anti-diagonals
- Used in many approaches
- With many variants

ANTI-DIAGONAL PARALLELISM



- Parallelism exists along anti-diagonals
- Used in many approaches
- With many variants

ANTI-DIAGONAL PARALLELISM



- Parallelism exists along anti-diagonals
- Used in many approaches
- With many variants

PARALLEL VERSION TAKE 3

- Anti-diagonal traversal!

```
For d in anti-diagonals:
#pragma omp parallel for
    For e in d.elements:
        (r,c) = convert_de_to_rc(d,e)
        F[r][c] = max(…)
```

		A	A	G
	0	0	0	0
A	0	①		
G	0	②		
C	0	③		

- No synchronization
- Easy parallelization
- Any problem?

PARALLEL VERSION TAKE 3

- Anti-diagonal traversal!

```
For d in anti-diagonals:
#pragma omp parallel for
    For e in d.elements:
        (r,c) = convert_de_to_rc(d,e)
        F[r][c] = max(…)
```

		A	A	G
	0	0	0	0
A	0	①		
G	0	②		
C	0	③		

- No synchronization
  - Implicit barrier per anti-diagonal
- Easy parallelization
  - Bad cache efficiency (duplicate accesses, false sharing, etc)

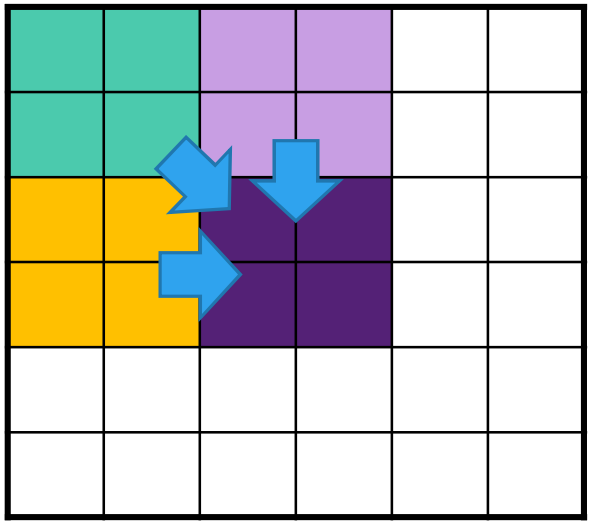
IDEAS

- A topic for research
- Address transforming
  - Like matrix transpose

		A	A	G
A		0,0	0,1	0,2
G		1,0	1,1	1,2
C		2,0	2,1	2,2

0,0	1,0	0,1	2,0
1,1	0,2	2,1	1,2
2,2	…		

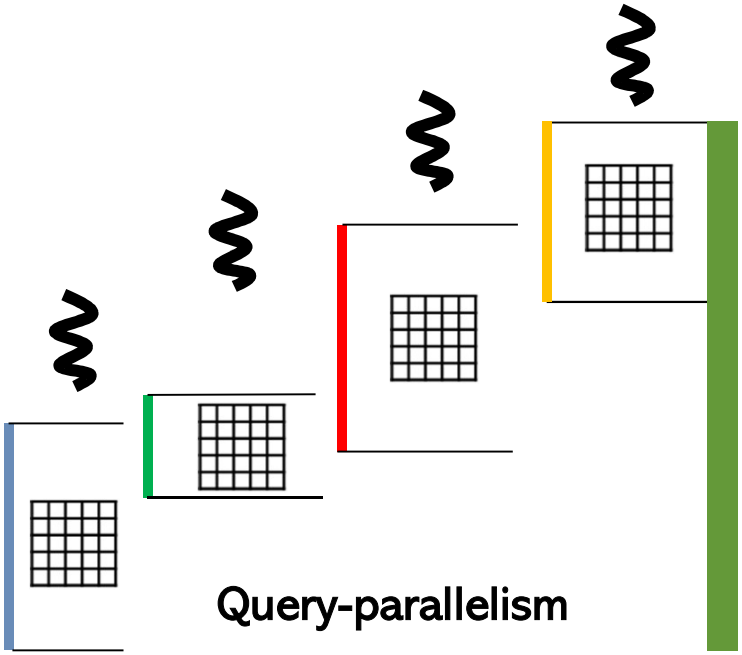
- Blockwise parallelization with job queuing
  - Wait until dependent blocks are ready
  - Allocate a block to a thread
  - Work distributed like a binary tree
  - Guided scheduling may be applied



GPU ALIGNMENTS 1 – GASAL2 (2019)

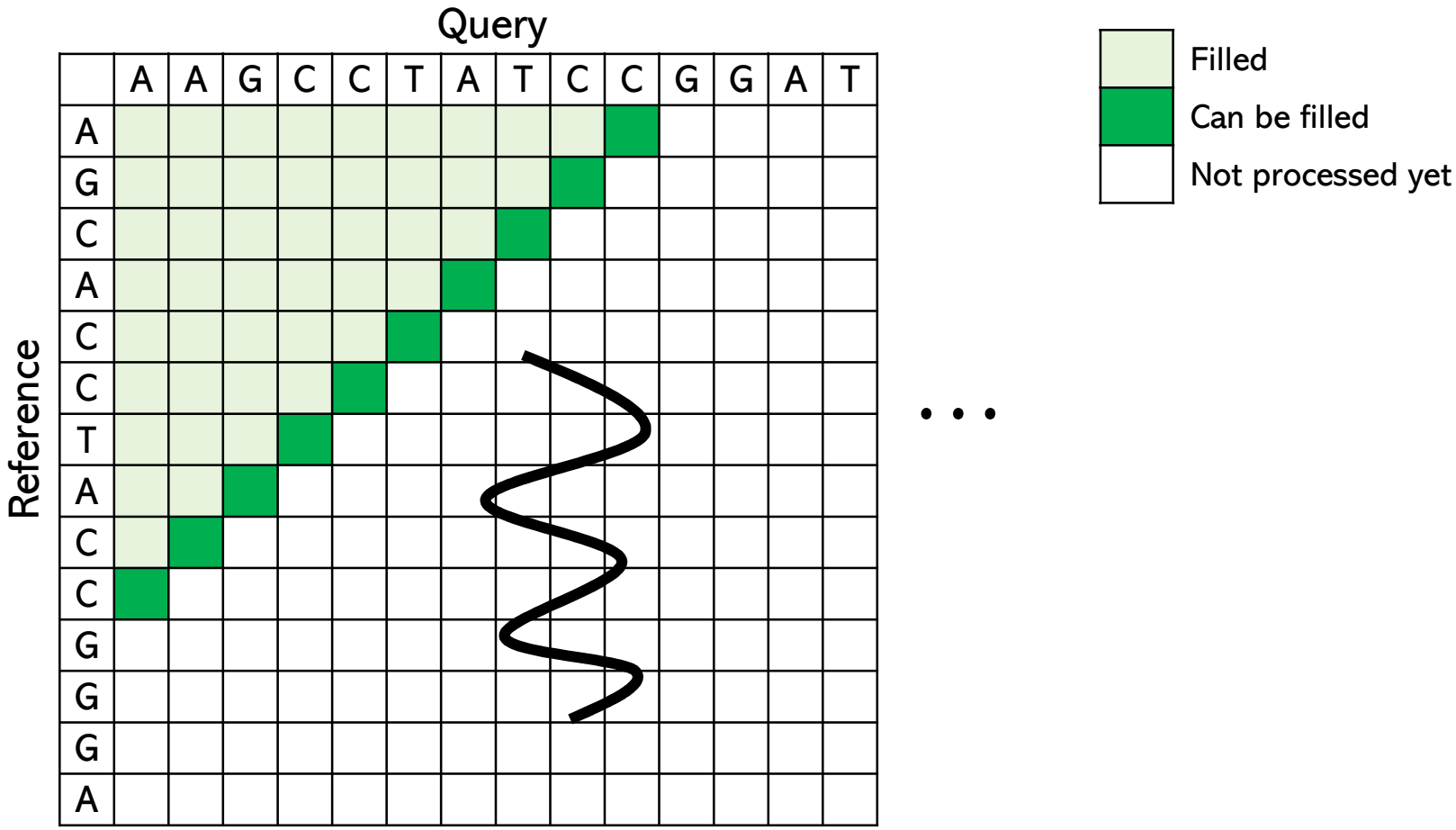
Read Mapping, method of aligning the reads against a known reference genome to **detect matches and variations.**

- Each read (query) requires filling one or more tables
- → **Query-parallelism**
- An easy way to write GPU code



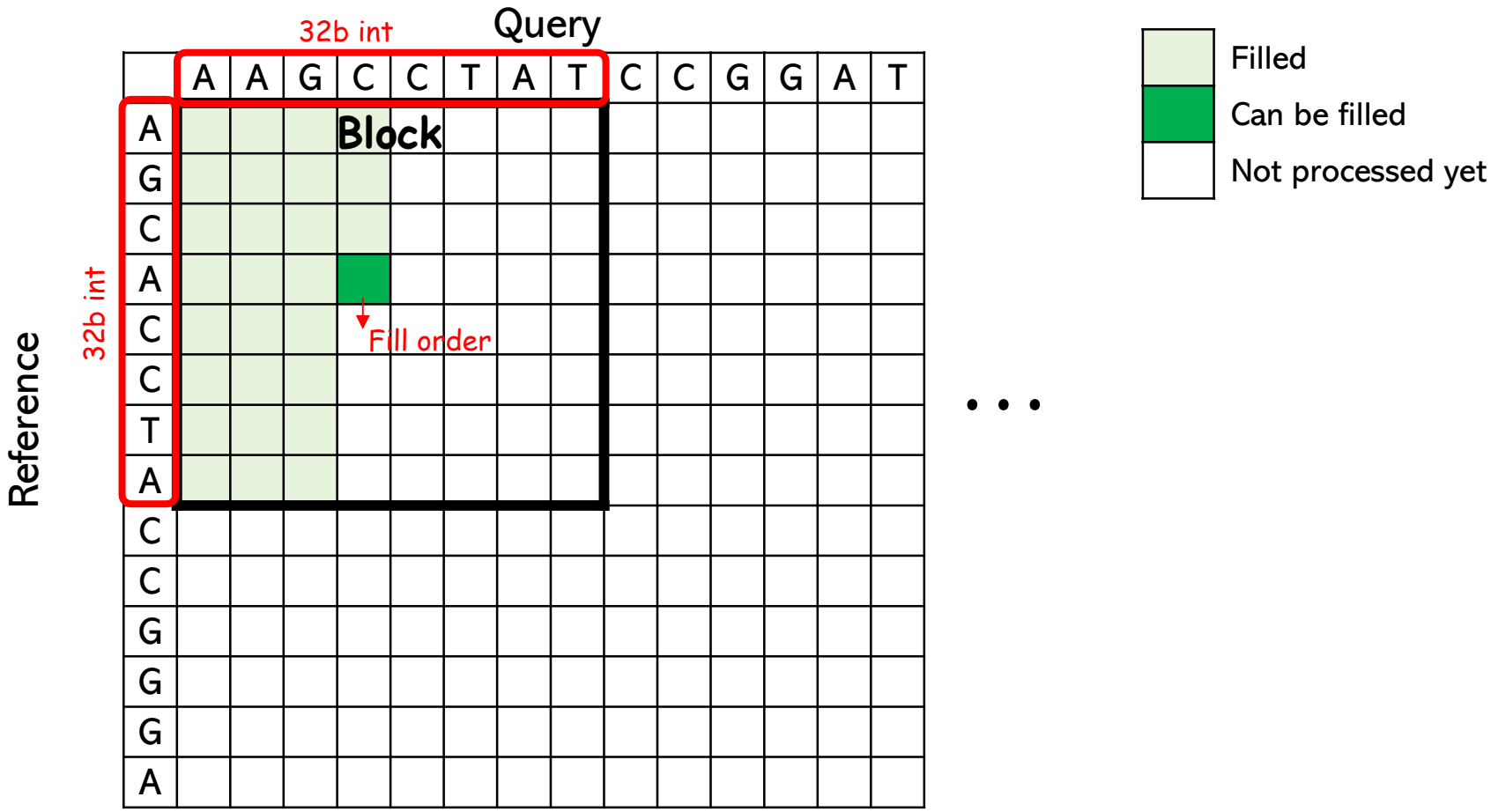
Reference Genome (of some other person)

GPU ALIGNMENTS 1 – GASAL2 (2019)



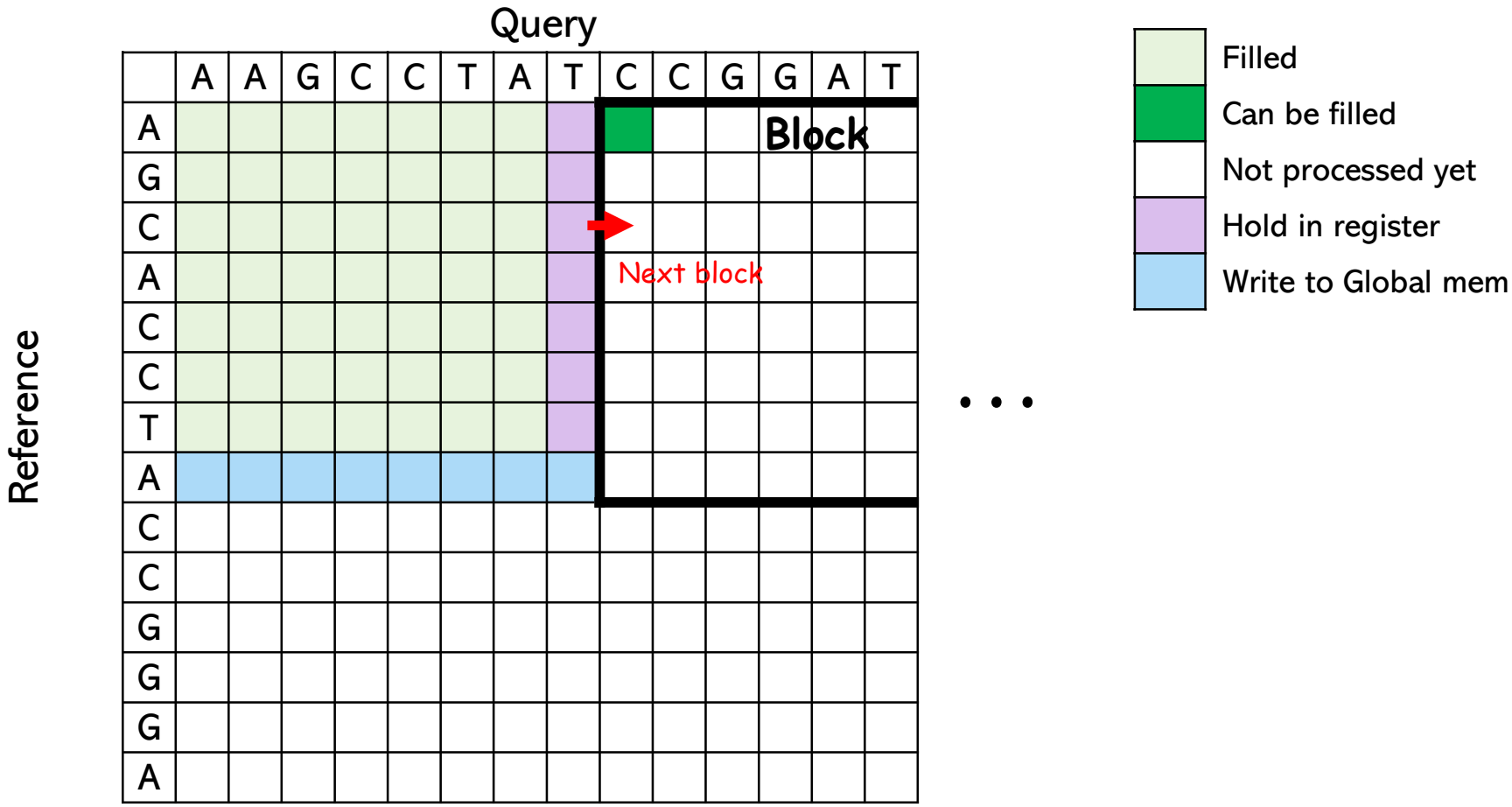
- The query / reference strings are read multiple times
  - Locality? Nope. We have only one thread per table
- Pack 8 literals into a 32b word
  - 4bit per literal (2bit was insufficient, because of the fifth literal `N' (unknown)

GPU ALIGNMENTS 1 – GASAL2 (2019)



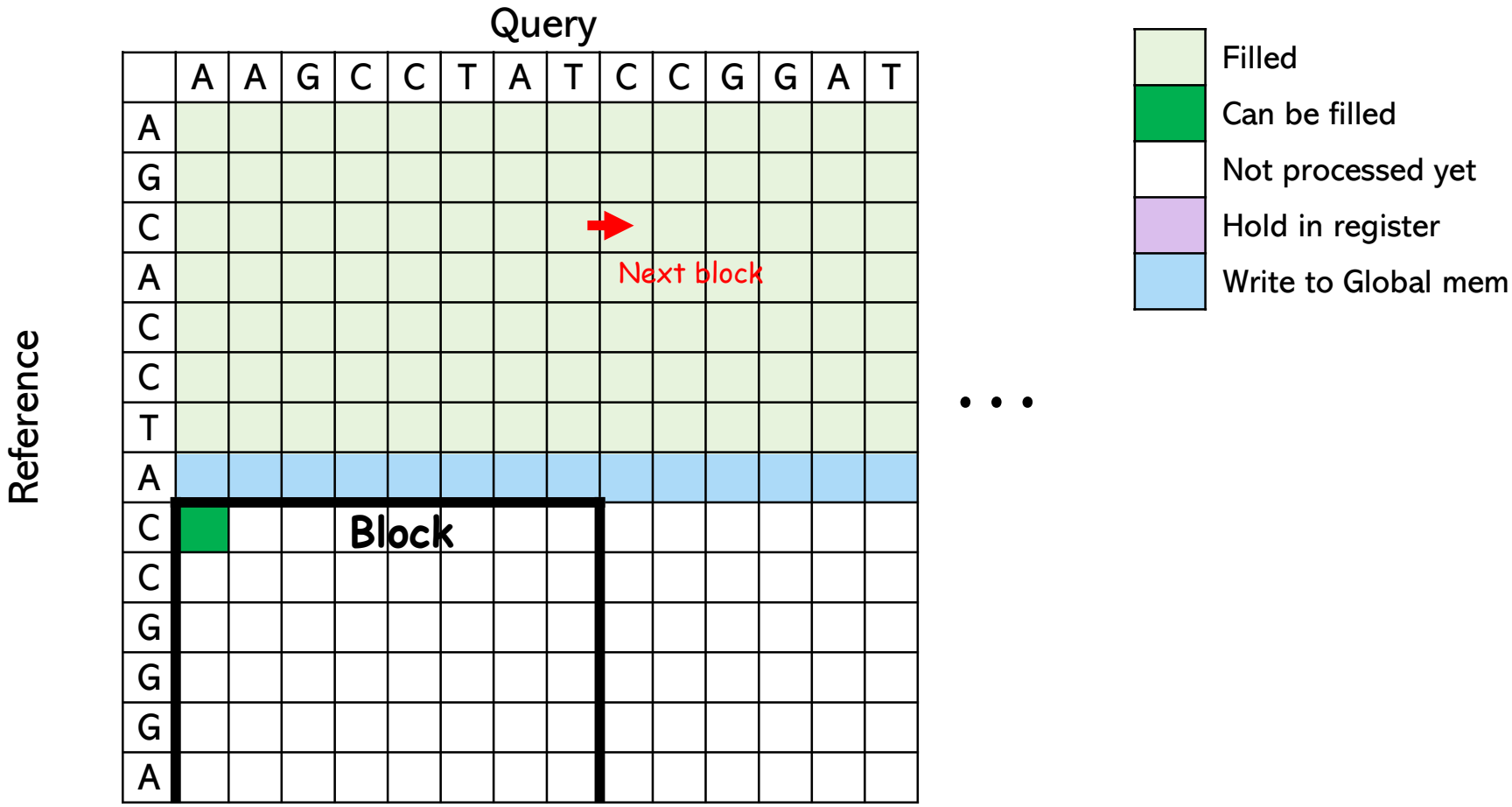
- 8x8 cells called a `block'
  - Fill in a sequential order within each block

GPU ALIGNMENTS 1 – GASAL2 (2019)



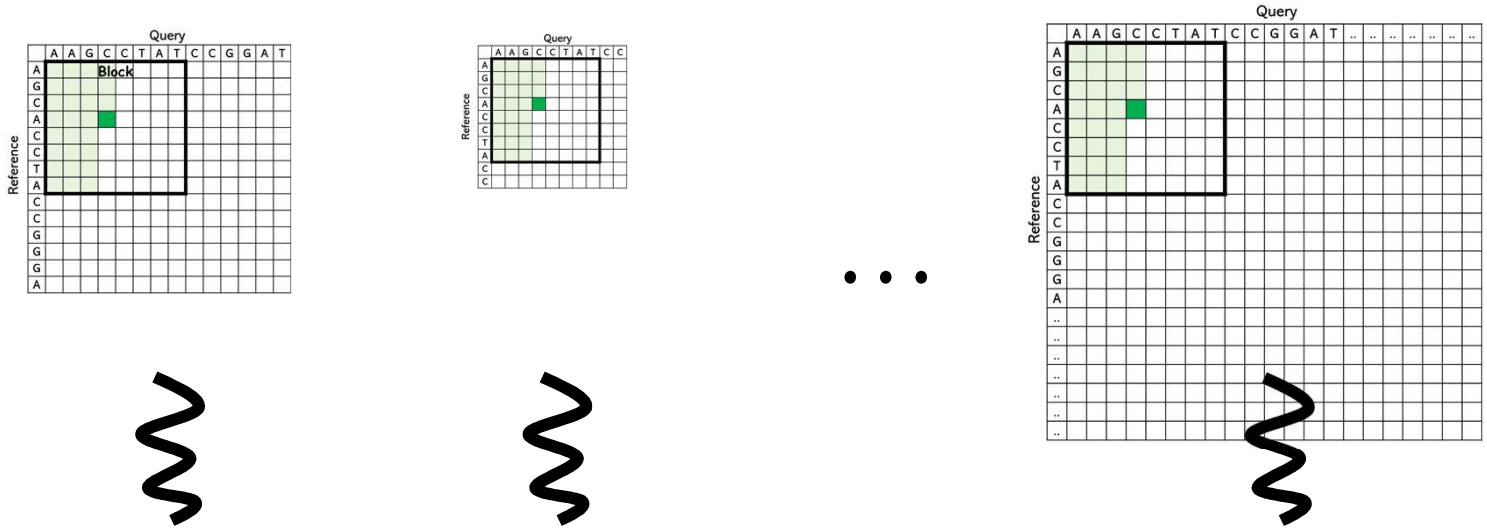
- When moving to the next block
  - Hold directly needed ones in the register
  - Write the rest into global memory

GPU ALIGNMENTS 1 – GASAL2 (2019)



- When moving to the next block
  - Hold directly needed ones in the register
  - Write the rest into global memory
    - Loaded when we come back

GPU ALIGNMENTS 1 – GASAL2 (2019)



(+) Easy parallelization

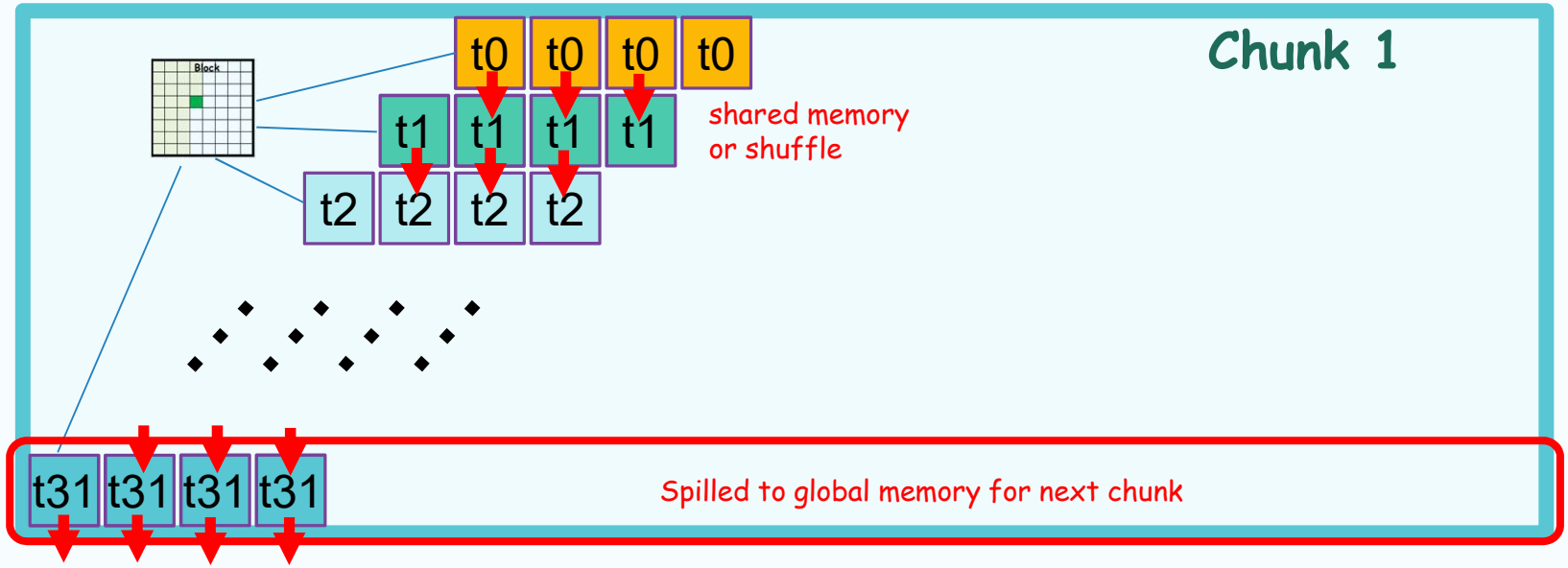
(-) No coalescing

(-) No shm locality

(-) Workload imbalance
(warp divergence)

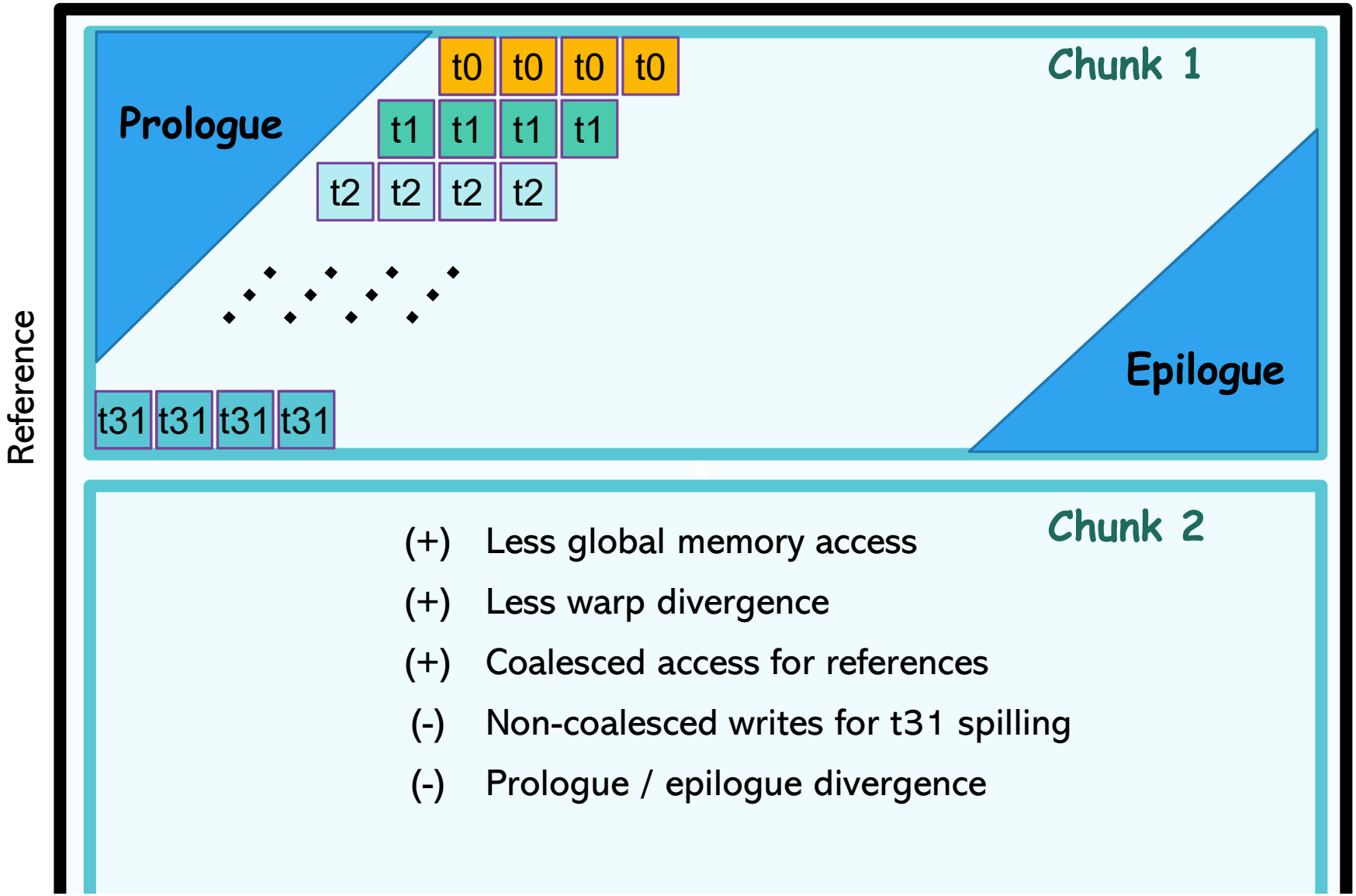
GPU ALIGNMENTS 2 – SALOBA (2022)

- Intra-query parallelism (use anti-diagonal)



GPU ALIGNMENTS 2 – SALOBA (2022)

- Intra-query parallelism (use anti-diagonal)



(+) Less global memory access

(+) Less warp divergence

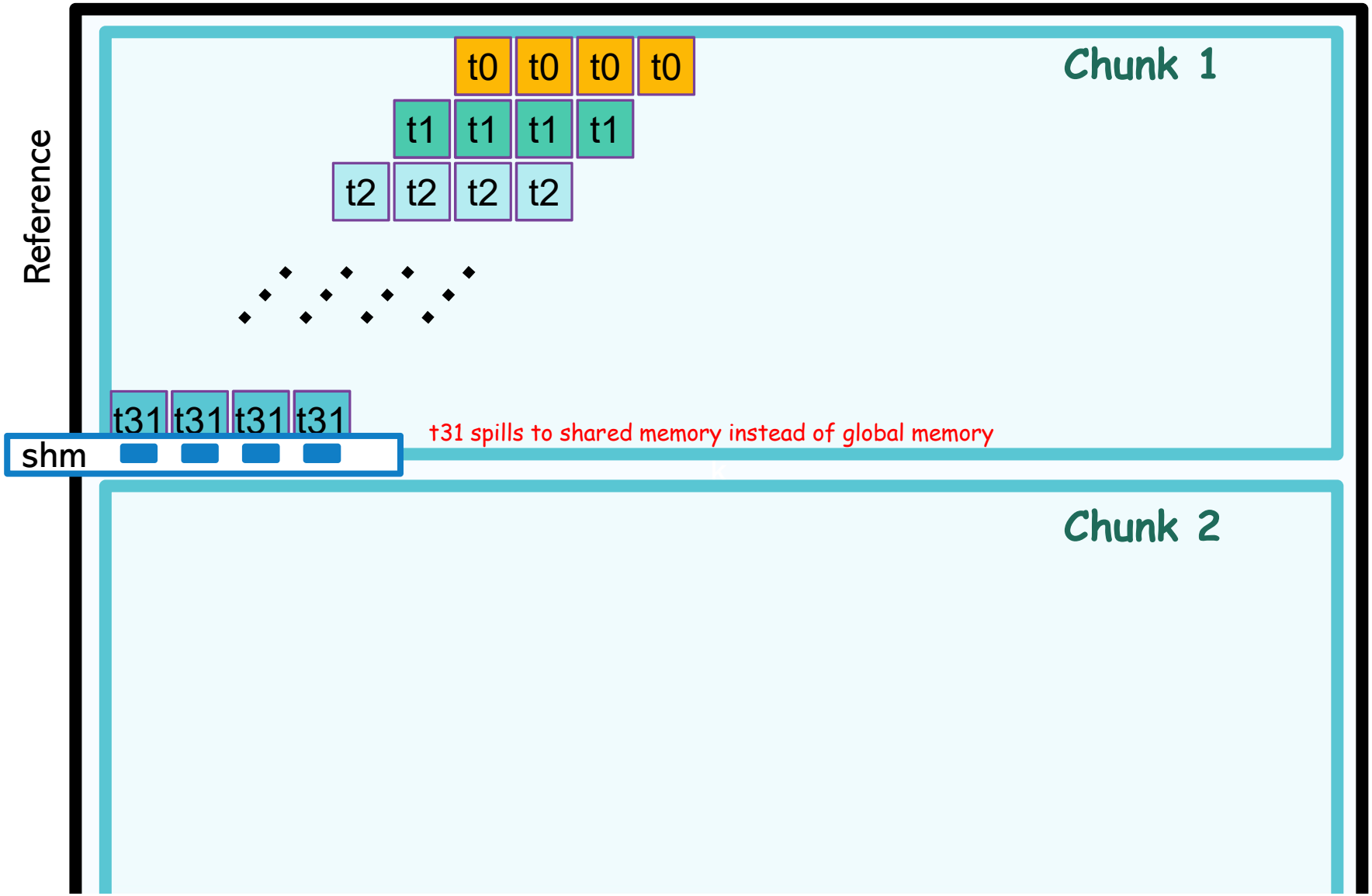
(+) Coalesced access for references

(-) Non-coalesced writes for t31 spilling

(-) Prologue / epilogue divergence

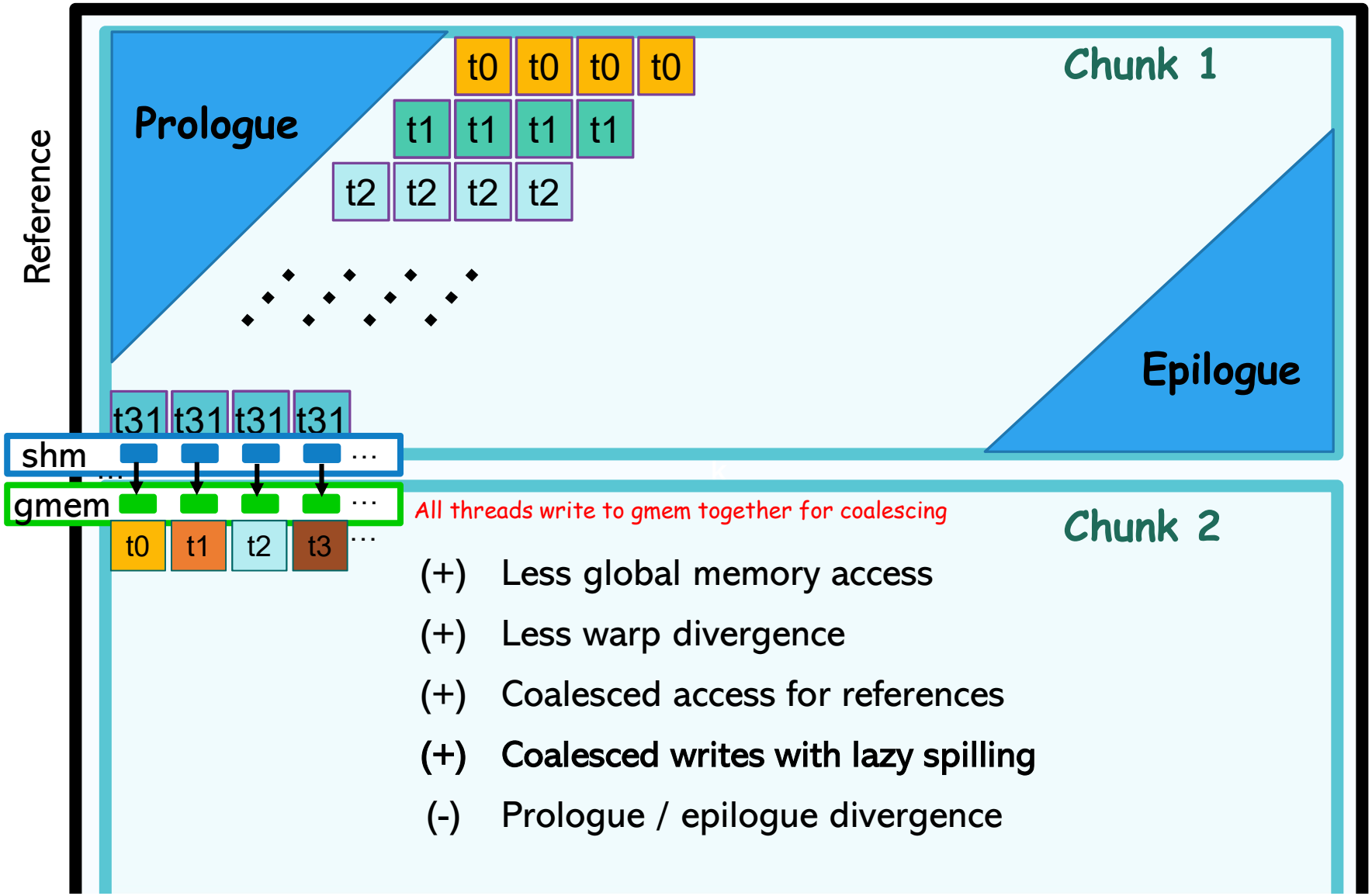
GPU ALIGNMENTS 2 – SALOBA (2022)

- Lazy spill for more coalescing



GPU ALIGNMENTS 2 – SALOBA (2022)

- Lazy spill for more coalescing



(+) Less global memory access

(+) Less warp divergence

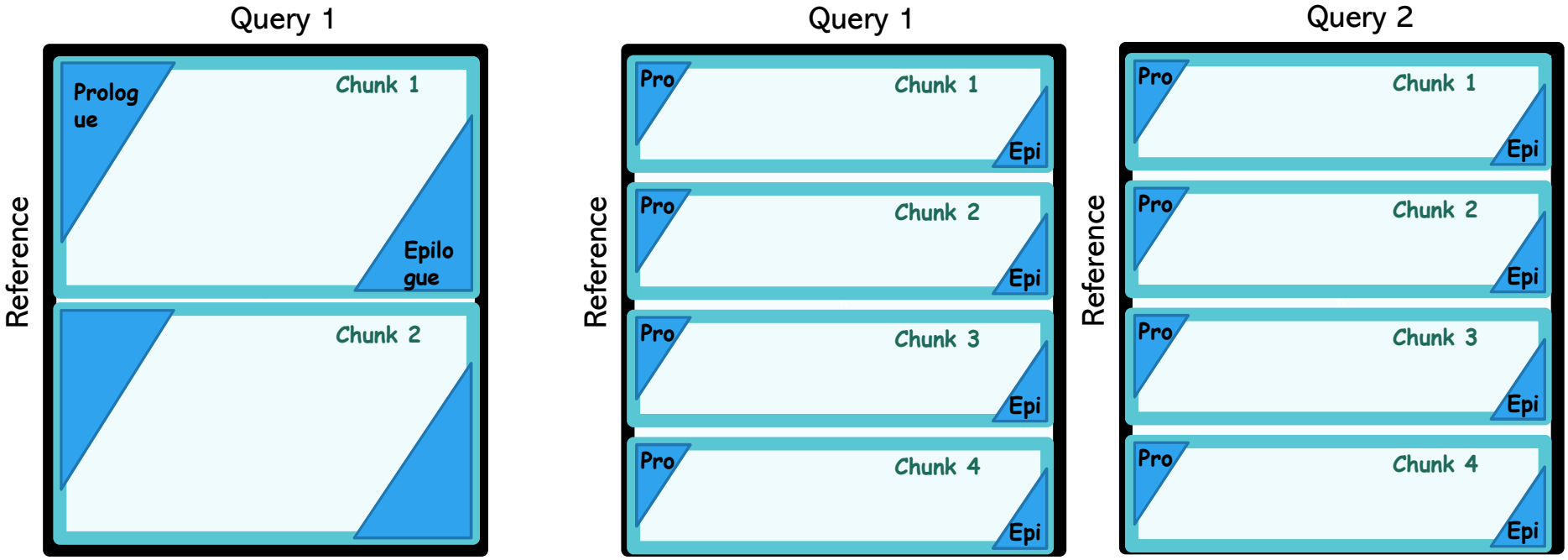
(+) Coalesced access for references

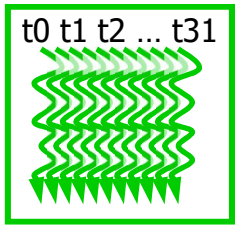
(+) **Coalesced writes with lazy spilling**

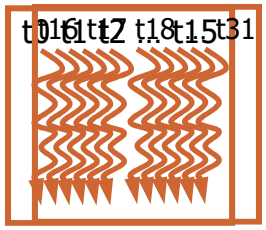
(-) Prologue / epilogue divergence

GPU ALIGNMENTS 2 – SALOBA (2022)

- Subwarp scheduling – prologue/epilogue size ∝ warp size



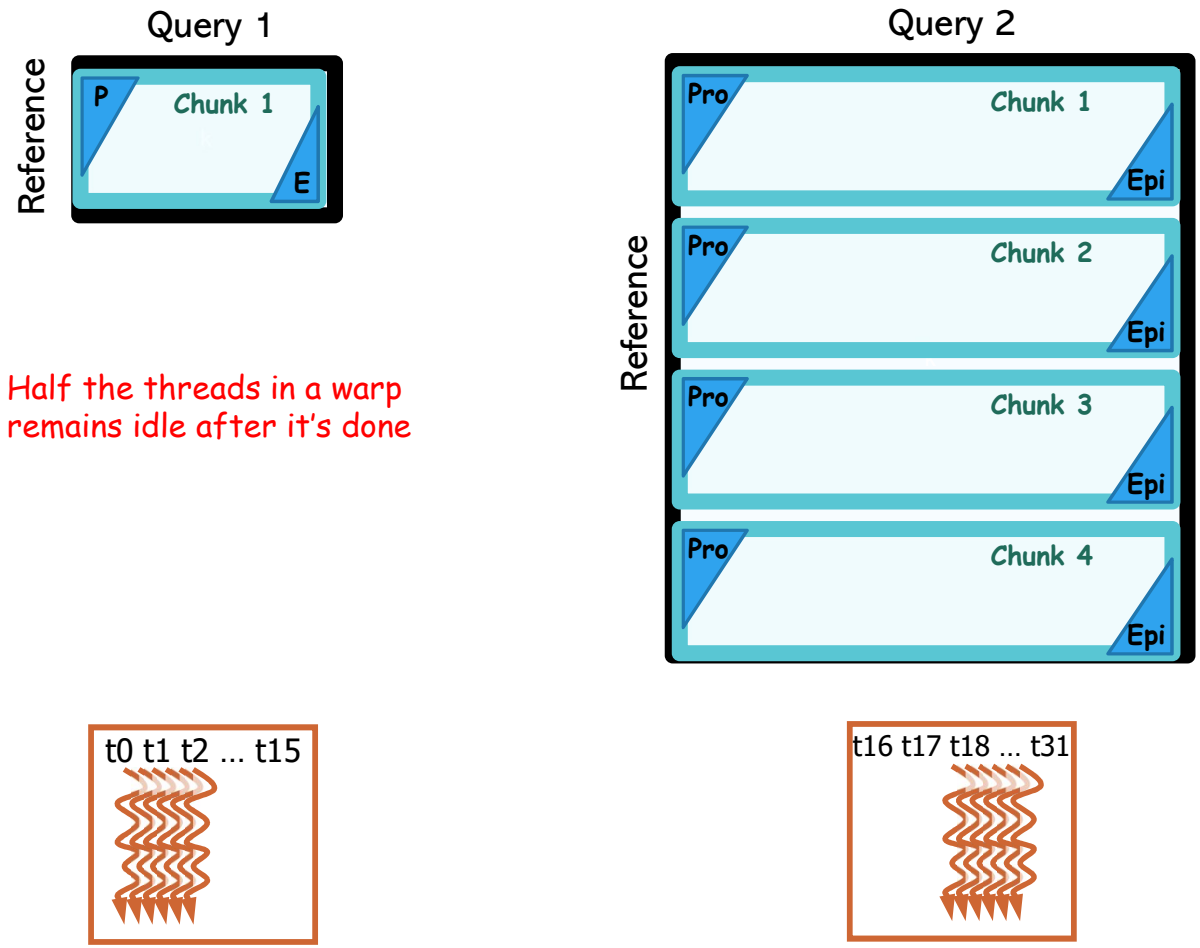




- Whole warp per query
  - Less inter-query divergence
  - Larger prologue/epilogue divergence
- Sub (partial) warp per query
  - More inter-query divergence
  - Smaller prologue/epilogue divergence

GPU ALIGNMENTS 3 – AGATHA (2023)

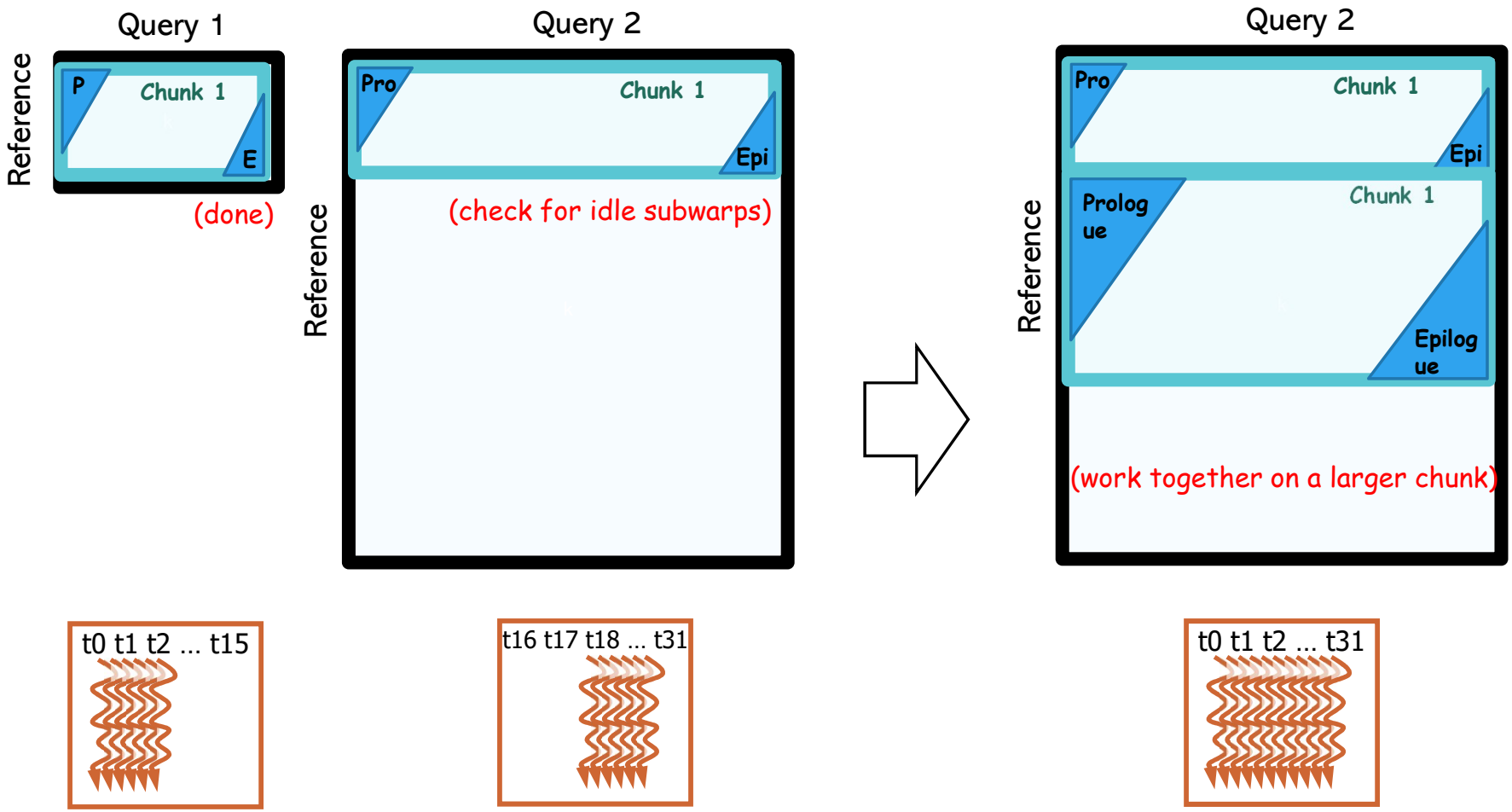
- Inter-warp divergence is larger when the worksize difference is large



Half the threads in a warp remains idle after it's done

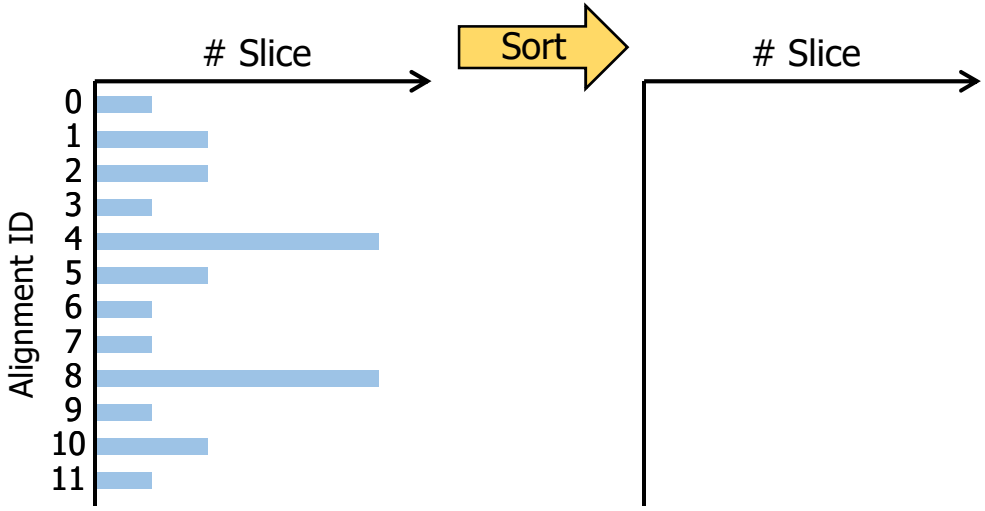
GPU ALIGNMENTS 3 – AGATHA (2023)

- Subwarp rejoining – Merge idle subwarps at the beginning of a chunk (~=Work stealing/spilling)



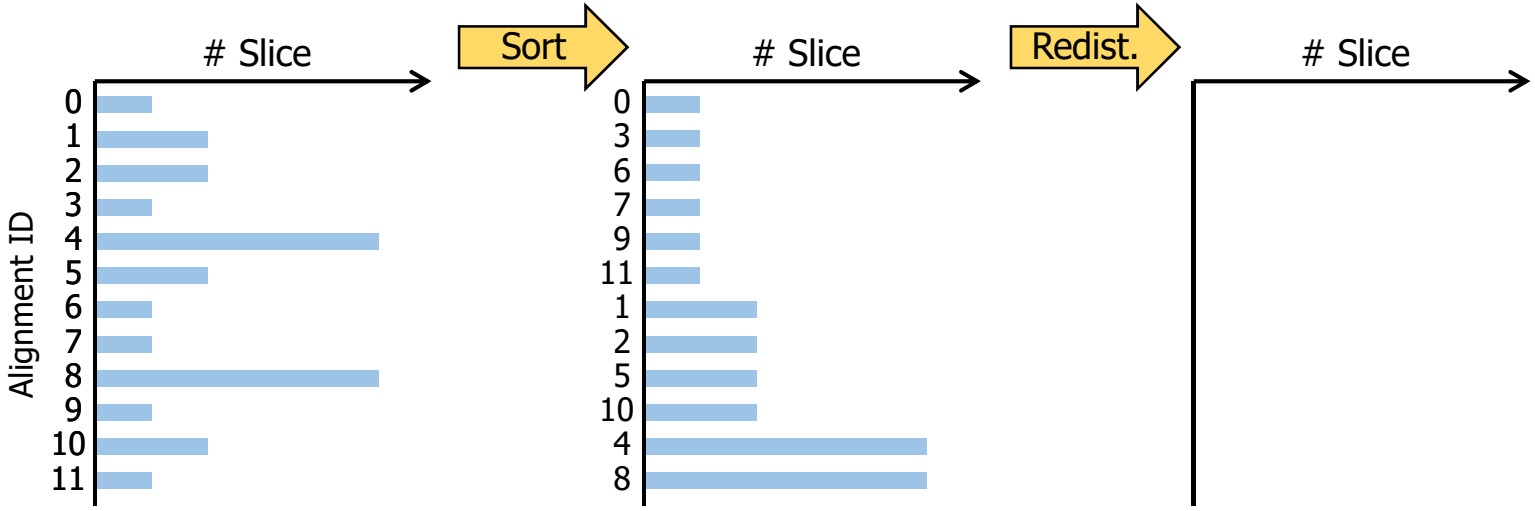
GPU alignments 3 – AGAThA (2023)

- **Uneven Bucketing:**
  Inter-warp workload balancing with sorting and work redistribution.
  - Target: Inter-warp workload imbalance



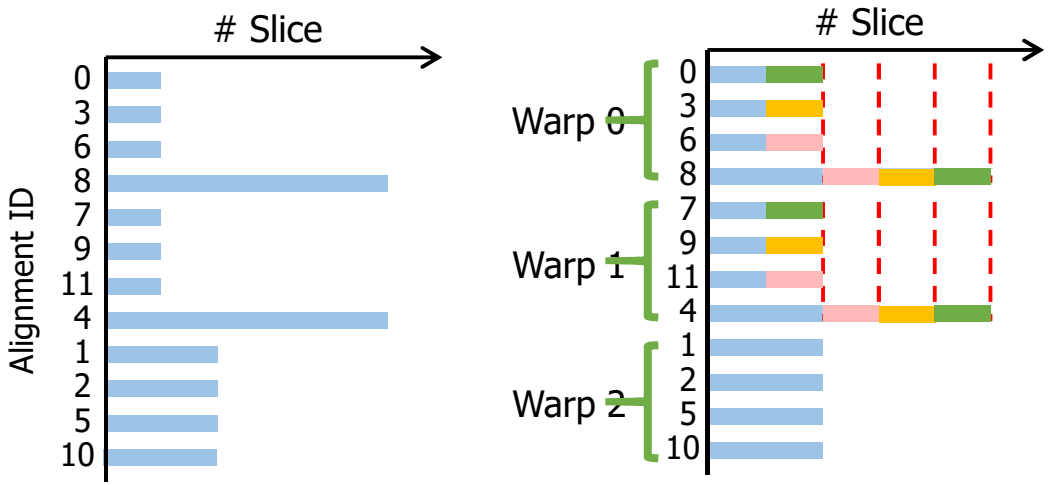
GPU alignments 3 – AGAThA (2023)

- **Uneven Bucketing:**
  Inter-warp workload balancing with sorting and work redistribution.
  - Target: Inter-warp workload imbalance



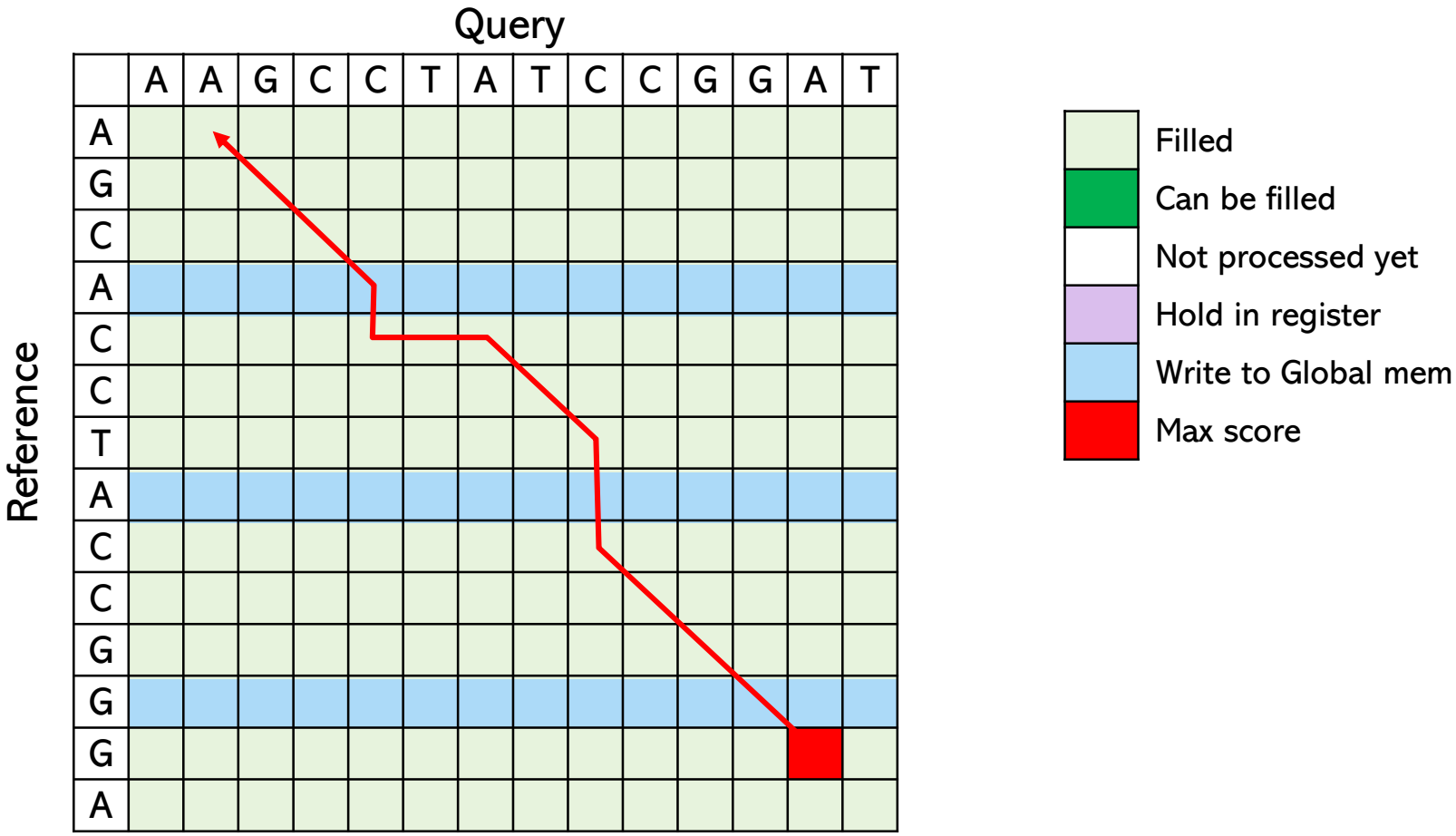
GPU alignments 3 – AGAThA (2023)

- **Uneven Bucketing:**
  Inter-warp workload balancing with sorting and work redistribution.
  - Target: Inter-warp workload imbalance



0. Uneven Bucketing.
1. Assign alignments to warps.
2. Align.
3. When alignment(s) terminate, Subwarp Rejoining.
4. Align until end.

GPU ALIGNMENTS 4 - S^3A (2025?)



- Intra-query parallelism with shm works well except for traceback
  - The green cells are not stored in the global memory
- For traceback, one of the below has to be done
  1. Drop the optimization using shared memory and write everything to gmem
  2. Write direction values per cell

  → Both add too many memory accesses

G3SA: Gridded Traceback

Challenge : Memory access is the bottleneck

		A	G	A	T
		→	→	→	→
G	↓	↘	↘	→	→
C	↓	↘	↓	↘	↘
T	↓	↘	↓	↘	↘
G	↓	↘	↓	↘	↓

Direction Matrix

1. Requires $O(N^2)$ memory space
2. Requires $O(N^2)$ memory access

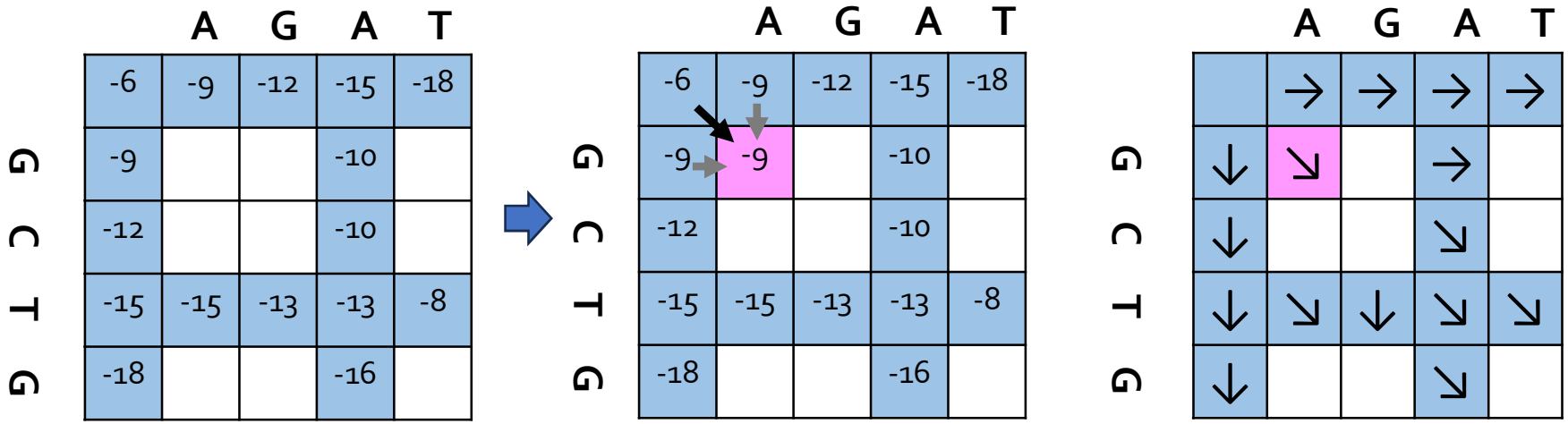
1. Degraded performance due to memory access overhead
2. Limited parallelism due to large memory footprint

G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

✓ **Key Observation : Score / Directions could be recomputed, given the boundary scores**

-> Store only the grid boundary scores in global memory



Scores stored in Global memory

Recompute Scores

Recompute Directions

G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

✓ Key Observation : Score / Directions could be recomputed, given the boundary scores

-> Store only the grid boundary scores in global memory

		A	G	A	T
	-6	-9	-12	-15	-18
G	-9			-10	
C	-12			-10	
T	-15	-15	-13	-13	-8
G	-18			-16	

Scores stored in Global memory

		A	G	A	T
	-6	-9	-12	-15	-18
G	-9	-9	-7	-10	
C	-12			-10	
T	-15	-15	-13	-13	-8
G	-18			-16	

Recompute Scores

		A	G	A	T
		→	→	→	→
G	↓	↘	↘	→	
C	↓			↘	
T	↓	↘	↓	↘	↘
G	↓			↘	

Recompute Directions

G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

✓ **Key Observation : Score / Directions could be recomputed, given the boundary scores**

-> Store only the grid boundary scores in global memory

		A	G	A	T
	-6	-9	-12	-15	-18
G	-9			-10	
C	-12			-10	
T	-15	-15	-13	-13	-8
G	-18			-16	

Scores stored in Global memory

		A	G	A	T
	-6	-9	-12	-15	-18
G	-9	-9	-7	-10	
C	-12	-12		-10	
T	-15	-15	-13	-13	-8
G	-18			-16	

Recompute Scores

		A	G	A	T
		→	→	→	→
G	↓	↘	↘	→	
C	↓	↘		↘	
T	↓	↘	↓	↘	↘
G	↓			↘	

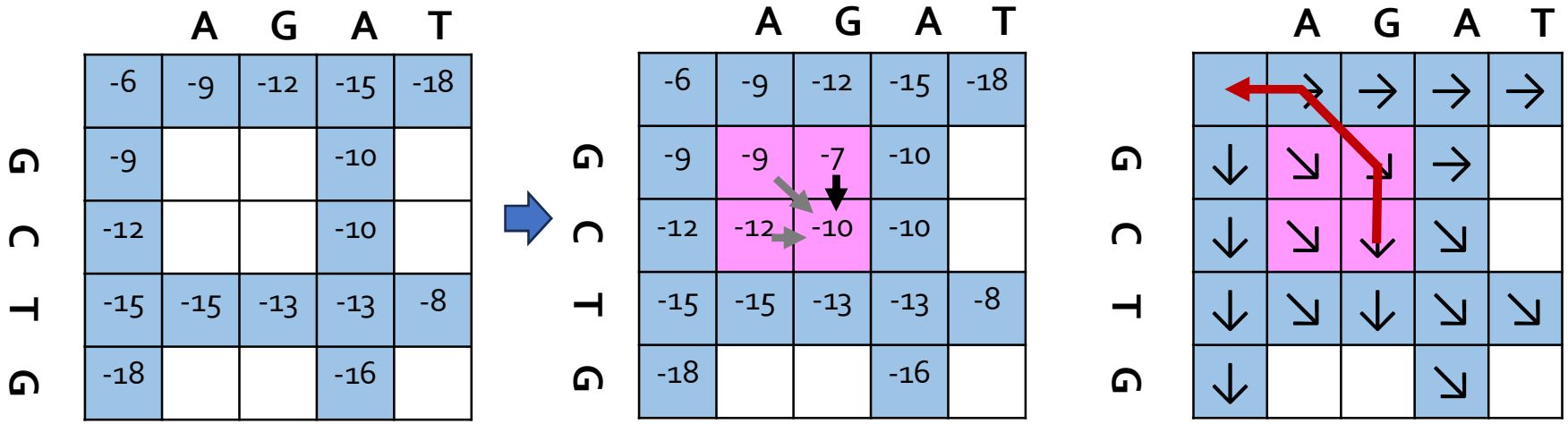
Recompute Directions

G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

✓ Key Observation : Score / Directions could be recomputed, given the boundary scores

-> Store only the grid boundary scores in global memory



Scores stored in Global memory

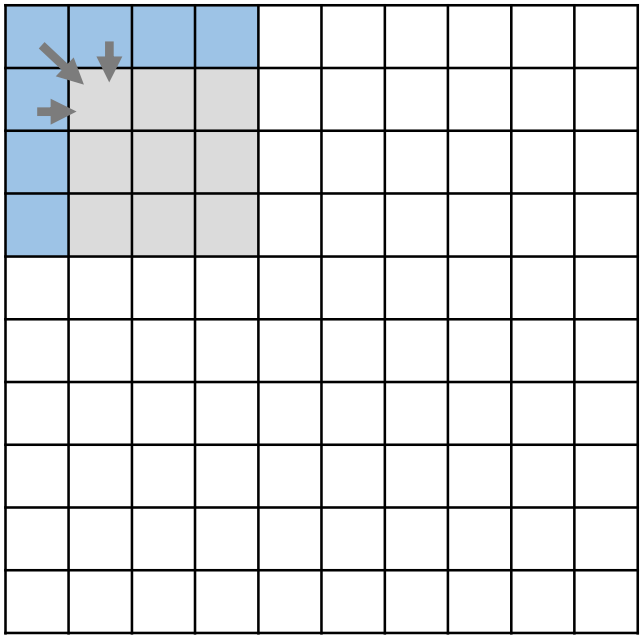
Recompute Scores

Recompute Directions

G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

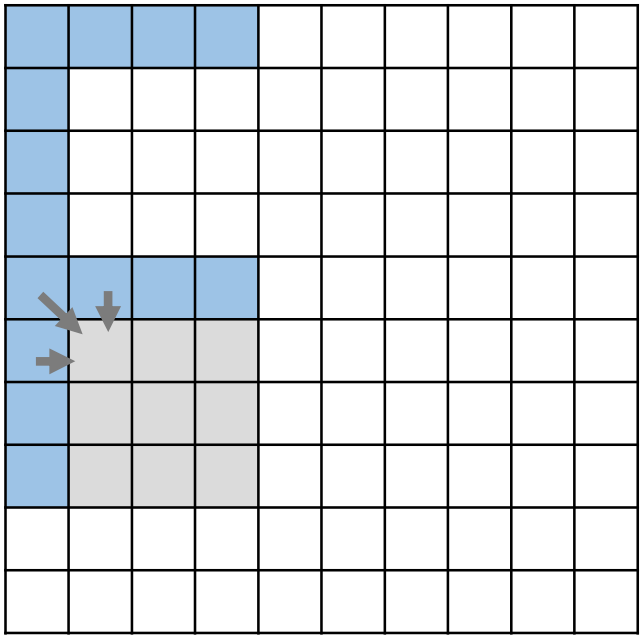
① Forward phase : compute score, store boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

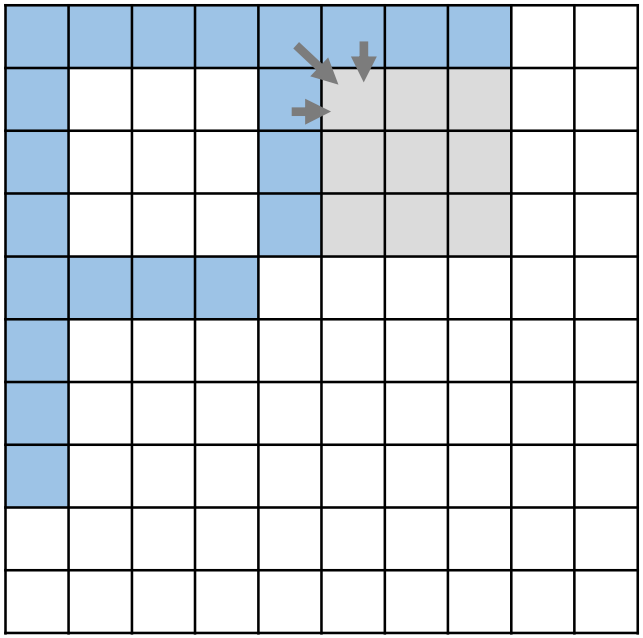
① Forward phase : compute score, store boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

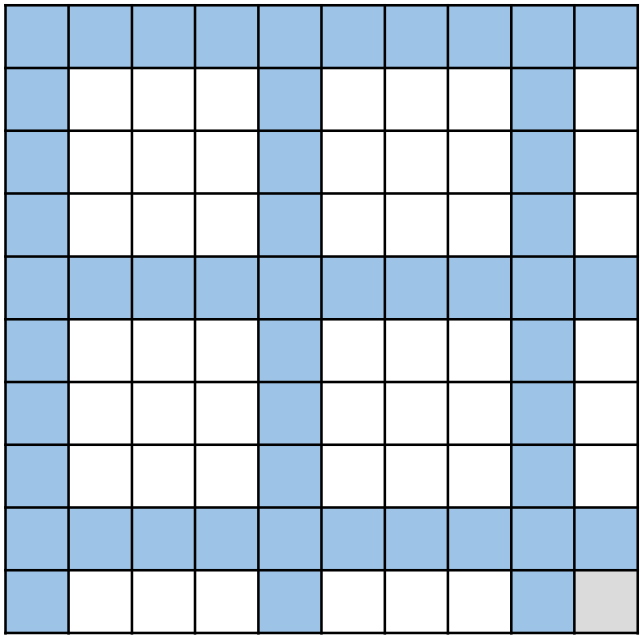
① Forward phase : compute score, store boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

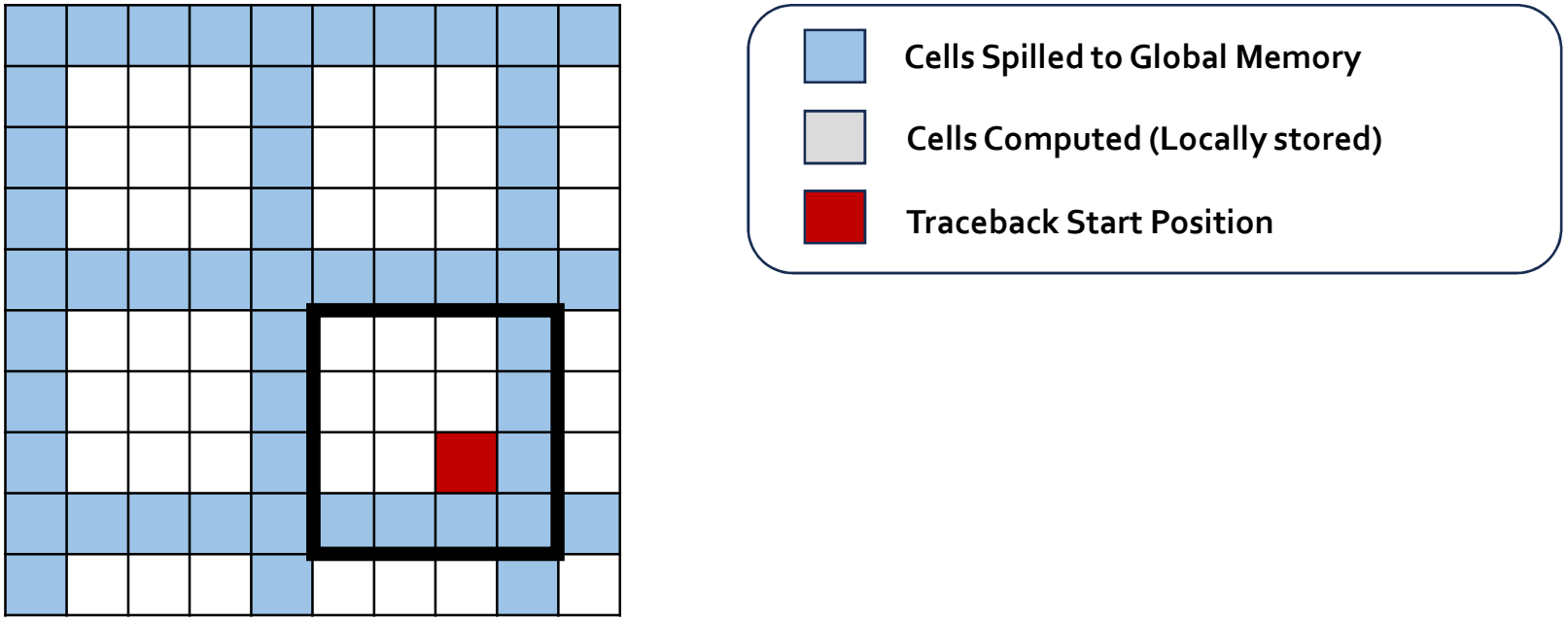
① Forward phase : compute score, store boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

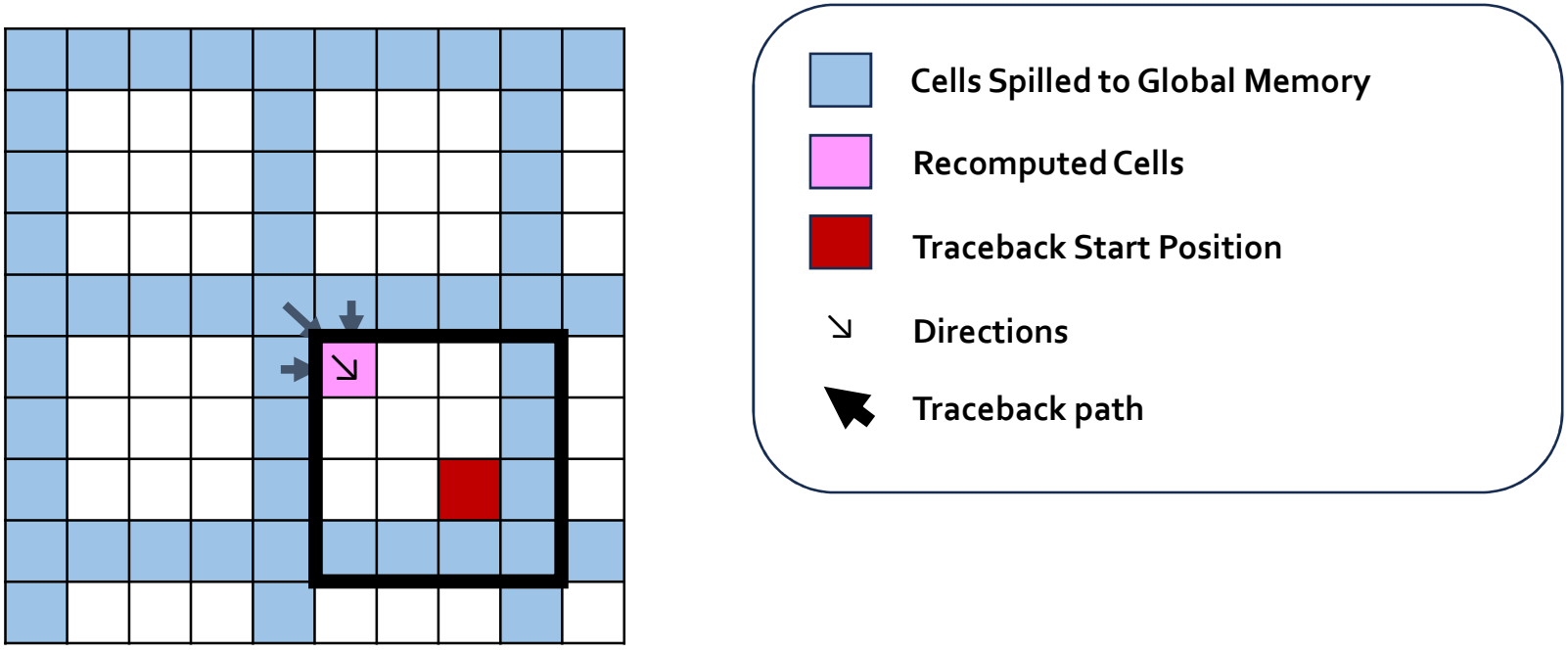
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

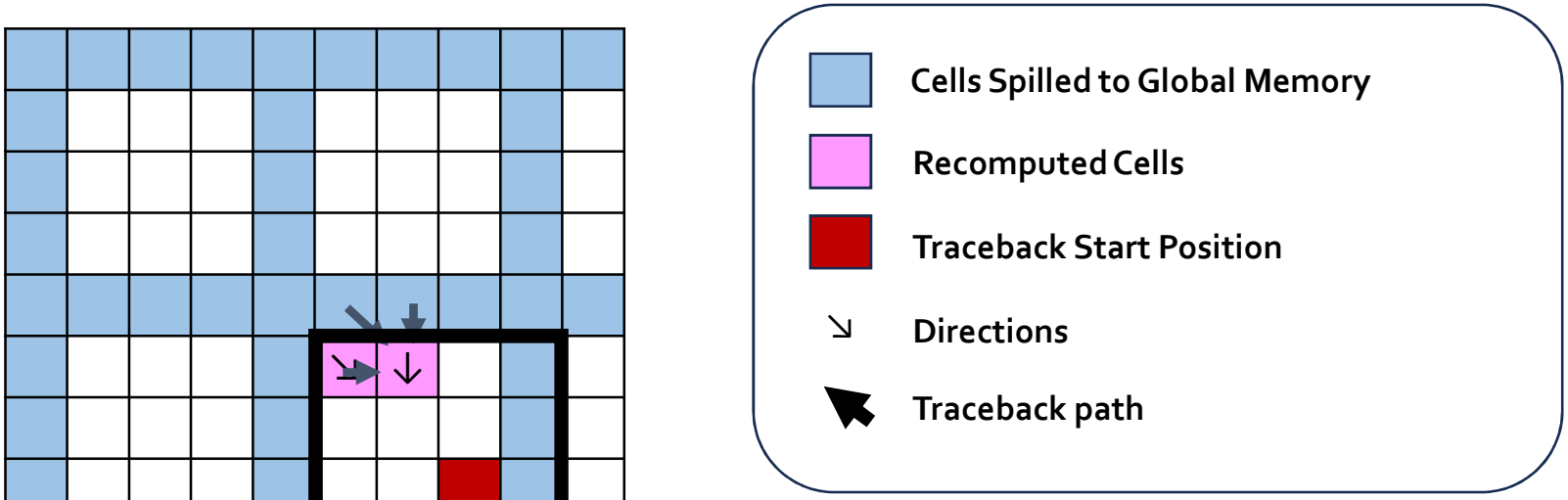
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

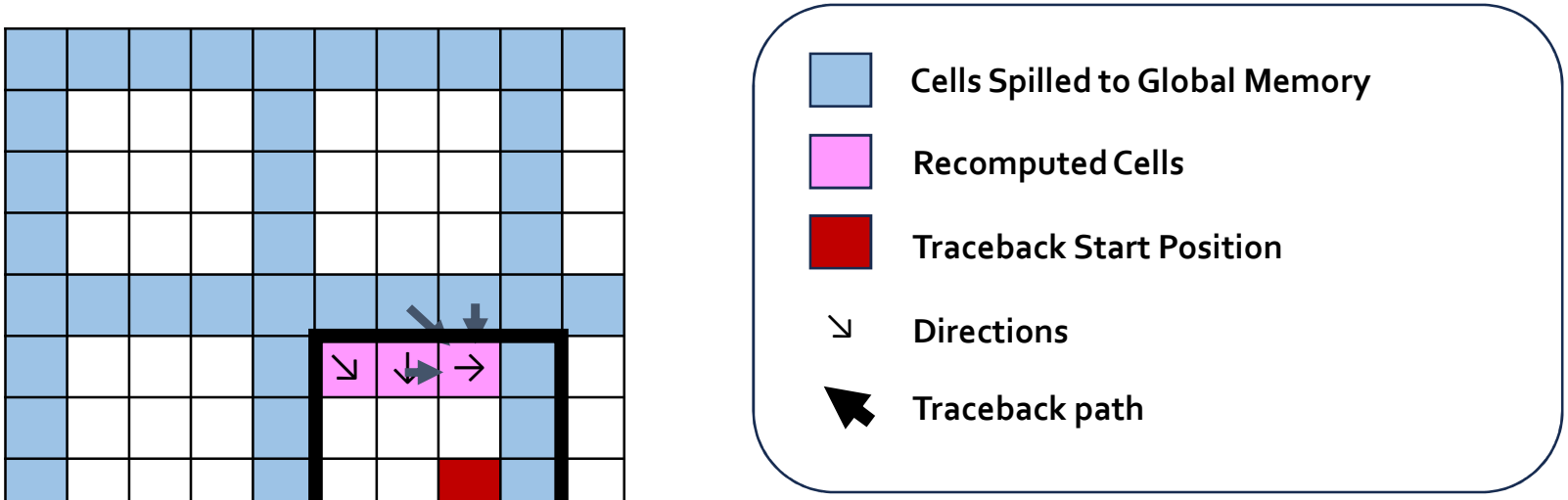
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

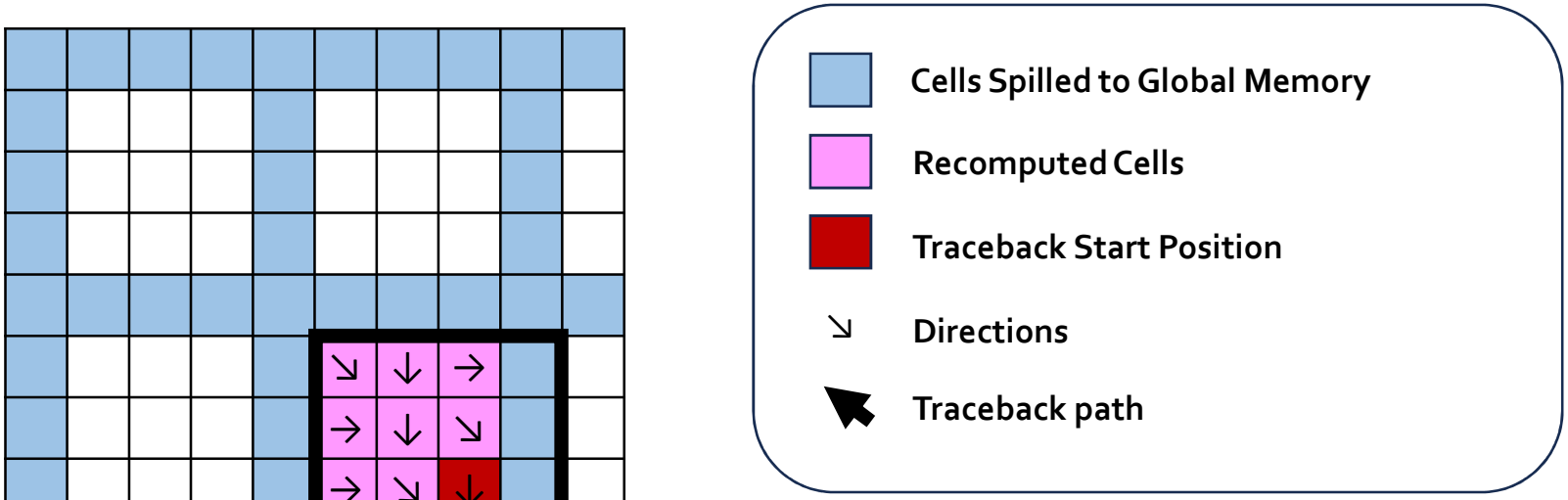
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

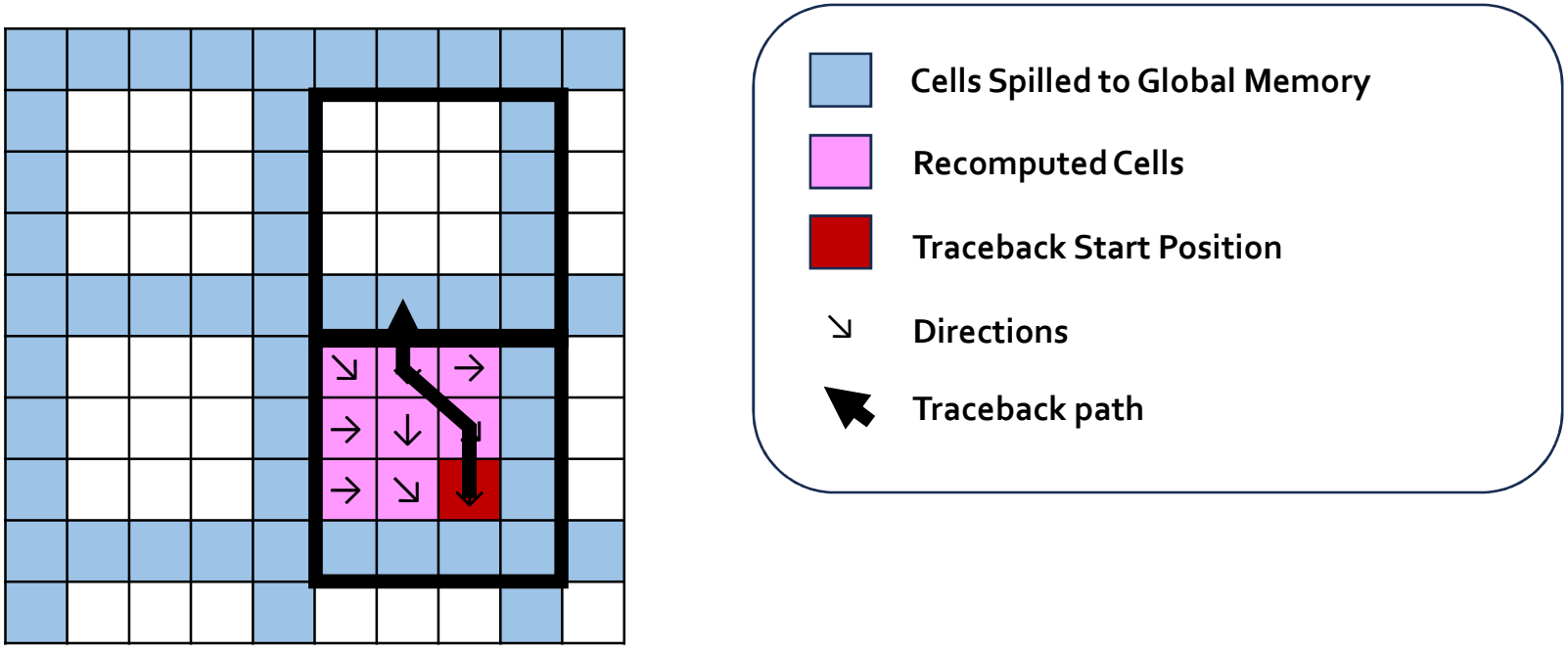
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

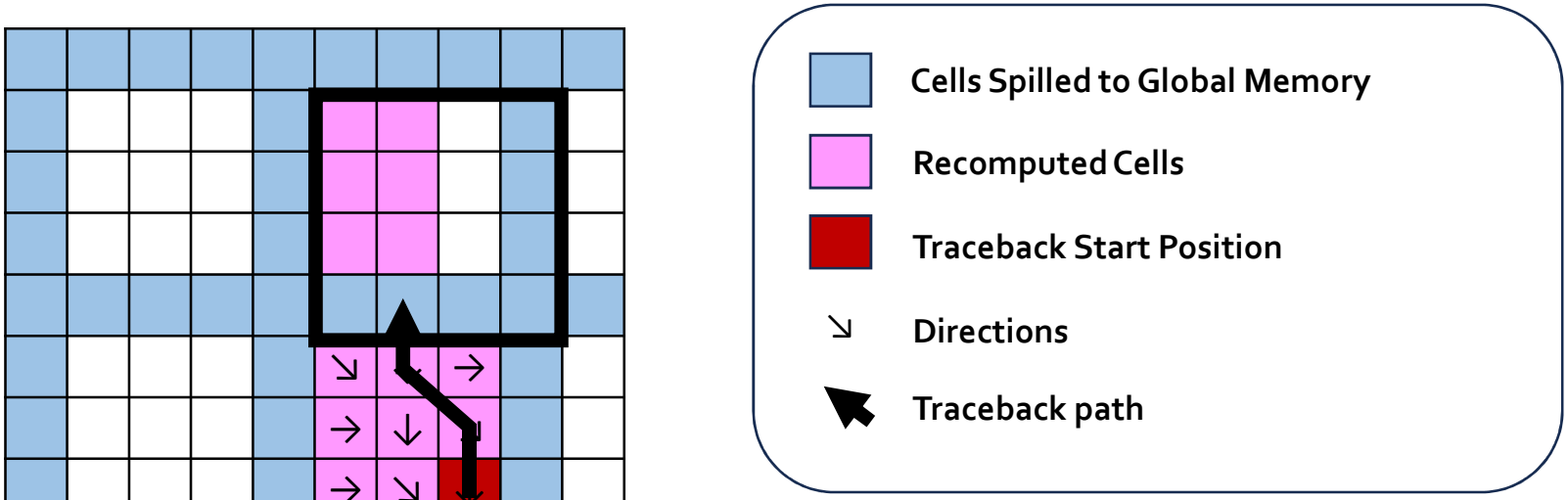
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

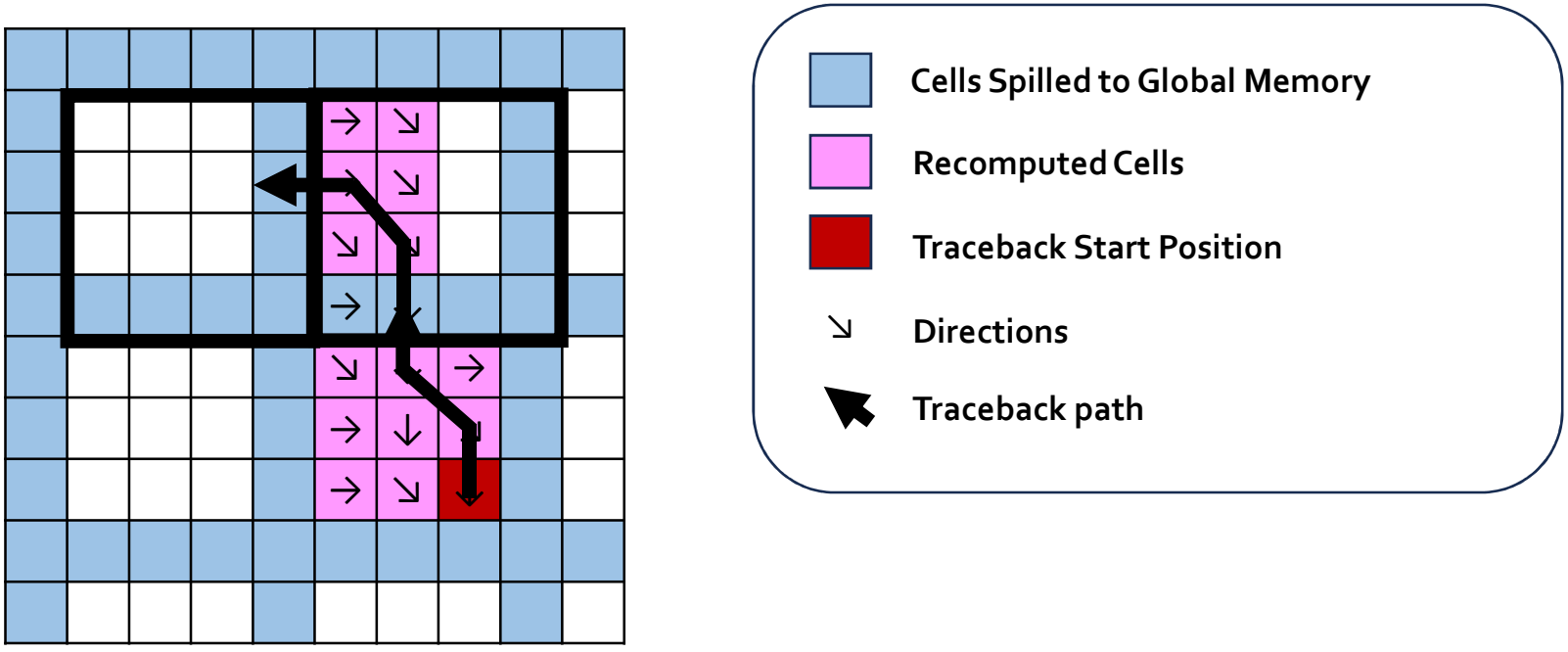
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

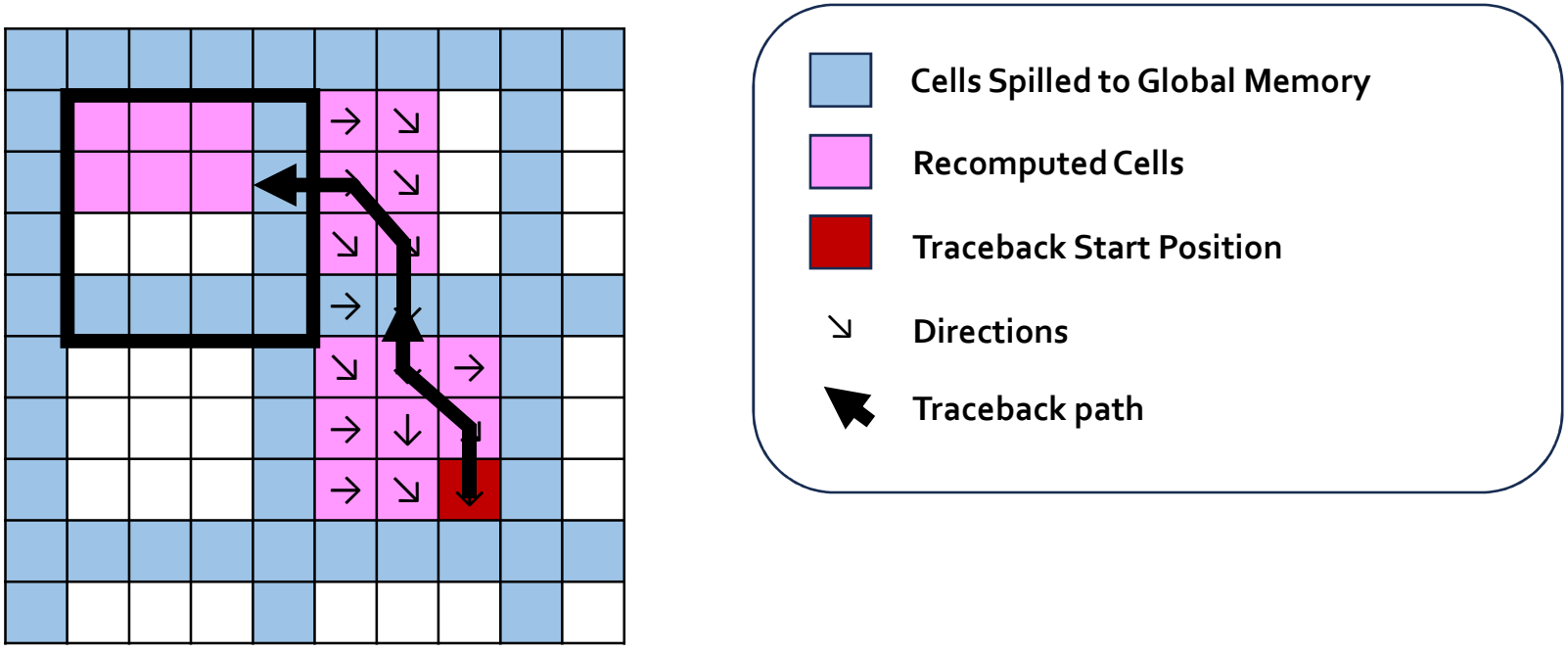
② Traceback phase : recompute scores & compute direction from boundary scores



G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

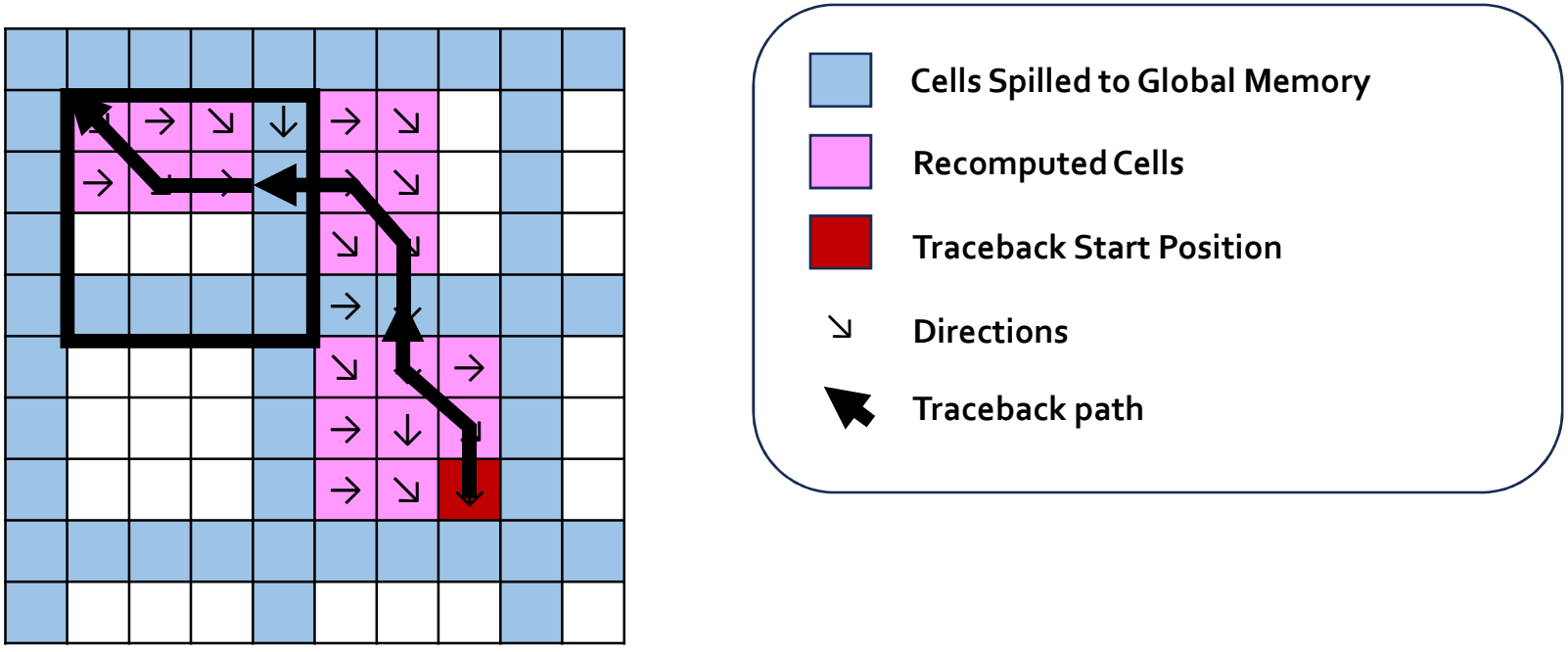
② Traceback phase : recompute scores & compute direction from boundary scores



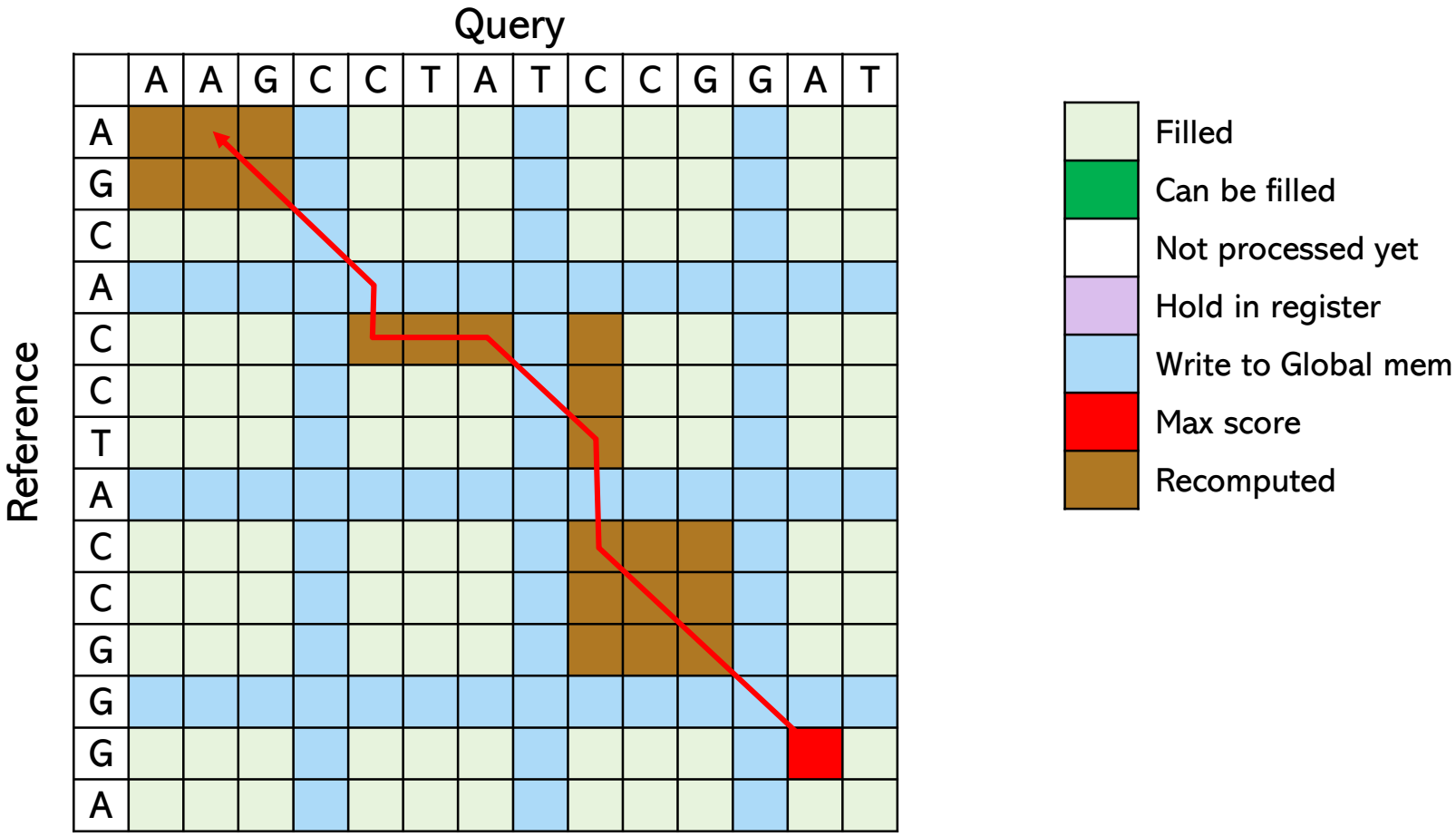
G3SA: Gridded Traceback

Gridded Traceback : Balancing the memory-computation tradeoff

② Traceback phase : recompute scores & compute direction from boundary scores



S³A (2025) – GRIDDED TRACEBACK



- Observation: most of the scores (or the directions) are not used in traceback
- Solution: Store grids, and
  - Small additional writes (better than writing everything)
  - Small additional recomputation
- Using query-level parallelism
  - Haven't found a way to utilize intra-query parallelism during traceback

SUMMARY

- DNA alignment – One interesting application for computer engineering
- Seemingly simple application requires a lot of optimizations
  - Many of those are what we have already covered
- Many research topics alive on several directions

- Remaining topics for study
  - CTA (Cooperative Thread Array)
    - Synchronize with other granularity than warp or threadblock
  - Vector types (similar to vector load in SIMD)
  - Multiple GPUs + Inter-GPU communication (NCCL, MPI)